



算法设计与分析

作者：Lewiz

时间：2023 年 6 月 27 日

微信公众号：EVO 空间

目录

1	算法引论	1
1.1	基本概念	1
1.2	算法复杂度	3
1.3	渐进分析	4
1.4	时间复杂度分析	6
1.4.1	计算迭代次数	7
1.4.2	递归方程	8
1.4.3	频度分析	9
1.4.4	复杂度分析的意义	9
2	数学基础与数据结构	10
2.1	递归方程求解	10
2.1.1	常系数线性同质递归方程	10
2.1.2	几类特殊的非同质递归方程求解	11
2.1.3	主定理 (Master Theorem)	13
2.1.4	递归树	13
2.1.5	代入法	14
2.2	数据结构	14
2.2.1	堆	14
2.2.2	并查集 (不相交集)	19
3	分治策略	22
3.1	基本思想	22
3.2	矩阵相乘	23
3.3	大整数相乘	24
3.4	排列问题	25
3.5	棋盘覆盖问题	25
4	动态规划	28
4.1	矩阵链相乘	28
4.2	平面凸多边形最优三角划分	29
4.3	0-1 背包问题	31
4.4	最长公共子序列问题	32
4.5	整数划分问题	33
4.6	集合划分	34
4.7	编辑距离	35

5	贪心策略	36
5.1	单源最短路径问题	36
5.1.1	Dijkstra 算法	36
5.2	最小生成树	36
5.2.1	Kruscal 算法	36
5.2.2	Prim 算法	37
5.3	哈夫曼编码	38
6	图遍历	39
6.1	深度优先遍历	40
6.2	广度优先遍历	41
7	回溯法和分支限界法	43
7.1	回溯法	43
7.1.1	图的 3 着色	44
7.1.2	n 皇后问题	44
7.1.3	总结	45
7.2	分支限界法	49
7.2.1	0-1 背包问题	50
7.2.2	分派问题	51
7.2.3	总结	52
8	随机算法	52
8.1	随机算法的特点	52
8.2	随机非重复采样问题	52

1 算法引论

1.1 基本概念

算法分析是计算机科学的一个核心领域，它研究如何设计和分析计算过程的有效性、效率和优化。以下是算法分析中的一些基本概念：

算法 算法是一组用于解决特定问题的**明确、有限**的步骤，这些步骤能够在**有限时间**内执行完毕并且产生确定的输出。一个好的算法应该是正确的、可靠的、高效的、易于理解的和可修改的。

1. 算法的四个性

- **确定性**
组成算法的每个指令都是清晰、无歧义的。
- **有限性**
每条指令的执行次数有限，执行每条指令的时间也有限。
- **输入**
零个或多个外部量作为输入。
- **输出**
至少产生一个量作为输出，它们与输入量之间存在特定的联系。

注：程序与算法的区别：程序是算法用某种程序设计语言的具体体现，可以不满足算法的有限性性质。

2. 如何评价一个算法？

从以下角度进行分析：

- **正确性**：算法是否能够正确地解决所需的问题？是否能够处理各种可能的输入情况？是否能够在合理的时间内给出正确的输出？
通常使用数学证明和形式化方法来证明算法的正确性。
- **效率**：算法是否具有高效性和可扩展性？是否能够在合理的时间内处理大规模数据？
- **可读性**：算法的实现是否容易理解和修改？是否易于维护和扩展？
- **健壮性**：算法是否能够处理不良输入和异常情况，例如无效输入、输入错误、崩溃和死锁等？
- **可行性**：算法是否可以在实际应用中运行？是否能够在目标硬件和软件环境中运行？是否需要额外的资源或依赖项？
- **优化度**：算法是否可以进一步改进和优化？是否存在更快、更节省空间、更可靠的实现方法？

题 1 ▶ 算法的复杂性与 _____、_____ 和 _____ 有关。

问题的规模、输入实例、算法本身

题 2 ▶ 理想情况下，希望能够使用符号 _____ 表示算法的时间复杂度。使用大 O 表示法表示算法的时间复杂度时，希望估计的上界的阶 _____ 越好。使用 Ω 表示算法的时间复杂度时，希望估计的下界的阶 _____ 越好。 Θ 、低、高。

题 3 ▶ 如果一个问题的阶可以表示成 _____，则可以考虑使用回溯法进行求解；经典的回溯法在进行搜索时，以 _____ 优先方式搜索解空间，并在搜索过程中用 _____ 函数避免无效搜索；分支限界法在搜索时使用 _____ 优先或是 _____ 优先展开搜索
定长的向量形式、深度、剪枝、广度、最小耗费（最大效益）

题 4 ▶ 动态规划的实质是 _____ 和 _____，是一种将问题实例分解为更小的、相似的子问题，并存储 _____ 以避免重复计算，来解决最优问题的算法策略。
分治、消除冗余、子问题的解

题 5 ▶ 分治算法一般分为三个步骤，即 _____、_____、_____
将原问题分解成若干子问题（分解）、分别求解子问题（解决）、将子问题的解组合成原问题的解（合并）

题 6 ▶ 设 $G = (V, E)$ 是连通无向带权图， (V, T) 是 G 的一个子图，并且 T 是一棵树，那么称 (V, T) 是 G 的生成树。如果 T 的权之和是所有生成树中最小的，那么称之为 _____。求解它的两种贪心算法是 _____ 和 _____
最小生成树、Kruscal 算法、Prim 算法

题 7 ▶ 分治算法与动态规划算法的不同之处在于：适合于分治法得到的子问题之间是 _____ 的，而适合于动态规划法求解的子问题之间是 _____ 的
相互独立、相互依赖

题 8 ▶ 经典的确定性算法必须满足 4 个特性：_____、_____、_____ 以及 _____。随机算法可以不满足其中的 _____ 特性。
确定性、有限性、输入、输出、确定性

题 9 ▶ 常用的算法设计策略包括 _____（填 5 个）
分治策略、动态规划、贪心策略、回溯法，分支限界法。

题 10 ▶ 快速排序是 _____ 设计策略的典型应用。理想情况下，快速排序的时间复杂度是 _____，平均情形下的时间复杂度为 _____，最差情形下的时间复杂度为 _____
分治、 $\Theta(n \log n)$ 、 $\Theta(n \log n)$ 、 $\Theta(n^2)$

题 11 ▶ 若定义两个顶点的几乎完全的二叉树高度为 1，则对于一棵高度为 k 的几乎完全的二叉树，其顶点个数 n 与 k 的之间的关系是 _____
 $2^k \leq n \leq 2^{k+1} - 1$

题 12 ▶ 算法是对问题 _____ 过程的准确描述，由 _____ 指令组成，这些指令能在 _____ 内执行完毕并产生 _____ 的输出
解决、有限条、有限时间、确定性

题 13 ▶ 堆是一种_____ 队列，可以用一个数组 H 来存放堆。将根节点存储在 $H[1]$ 中，那么，对于节点 $H[i]$ ，它的左右孩子节点（如果有的话）一定存放在_____ 和 _____ 中。 $H[i]$ 的父节点如果不是根节点，则存储在_____ 中

优先、 $H[2i]$ 、 $H[2i + 1]$ 、 $H[\lfloor i/2 \rfloor]$

题 14 ▶ 有两种常用的图遍历方法：_____ 和 _____

深度优先遍历、广度优先遍历

题 15 ▶ _____ 策略总是做出在当前看来最好的选择，并不从整体上考虑最优，所做出的的每一步选择只是局部意义上的最优选择，因而效率_____。使用贪心策略 _____ 保证得到问题的最优解。

贪心、较高、不一定

题 16 ▶ _____ 排序算法和 _____ 排序算法是两种比较典型的分治策略算法

快速、归并

题 17 ▶ 规模为 n 的 0-1 背包问题解空间大小是_____； n 个顶点的 3 着色问题的解空间大小是_____； n 皇后问题的解空间大小是_____

2^n 、 3^n 、 n^n

题 18 ▶ 最小生成树_____ 唯一

不是

题 19 ▶ 一个问题的解可以表示为定长的解向量形式 $(x_1, x_2, \dots, x_n)$ ，对于每个分量 x_i 取值的限定是_____ 约束、对不同分量之间的限定为_____ 约束

显式、隐式

题 20 ▶ 对一个问题实例，解向量满足_____ 约束的所有 n 维向量构成了该问题的一个_____

显式、解空间

1.2 算法复杂度

算法的复杂度是指算法运行时需要耗费计算机资源的量，需要的时间称为**时间复杂度**，需要的空间称为**空间复杂度**。

一个算法复杂度的高低体现在运行该算法所需要的计算机资源上，所需资源越多，我们就认为该算法的复杂度越高；反之，所需要的资源越少，我们就说该算法的复杂性低。

算法的复杂度与三个方面有关：

- 算法本身 A
- 问题规模 n
- 输入数据 I

注：实践表明：可操作性最强、最有实际价值的是算法最坏情形下的时间复杂度。自此，如不作特殊说明，使用 $T(n)$ 表示给定算法在最坏情形下的时间复杂度。

1.3 渐进分析

在渐进分析中，通常使用大 O 表示法、大 Ω 表示法、大 Θ 表示法和小 o 表示法来描述算法的渐进复杂度。以下是它们的定义和表示方式：

假设 $f(n)$ 和 $g(n)$ 是定义在正数集上的正函数，具体而言， $f(n)$ 表示算法的执行时间或空间复杂度， $g(n)$ 表示一个与 $f(n)$ 相关的函数，它可以是一个具有相同或更高阶的函数。通常， $g(n)$ 的形式可以是 n 、 n^2 、 $n \log n$ 等常见的函数形式。设 $f(n)$ 的阶为 a ， $g(n)$ 的阶为 b

大 O 表示法 对于函数 $f(n)$ 和 $g(n)$ ，如果存在常数 C 和 n_0 ，使得 $f(n) \leq C \cdot g(n)$ 对于所有 $n \geq n_0$ 都成立，则记作 $f(n) = O(g(n))$ 。

也就是说，函数 $f(n)$ 的增长速度不超过 $g(n)$ ，并且称 $f(n)$ 的阶不高于 $g(n)$ 的阶。则 $a \leq b$ 。

例如：

1. $f(n) = 2n^2 + 3n + 1$, $g(n) = n^2$ 。我们有 $f(n) = O(g(n))$ ，因为存在一个常数 $c = 3$ ，使得 $2n^2 + 3n + 1 \leq 3n^2$ 对于所有 $n \geq 1$ 都成立。因此， $f(n)$ 的增长速度不超过 $g(n)$ 的增长速度。
2. $f(n) = n^3 + 2^n$, $g(n) = 2^n$ 。我们有 $f(n) = O(g(n))$ ，因为存在一个常数 $c = 2$ ，使得 $n^3 + 2^n \leq 2 \cdot 2^n$ 对于所有 $n \geq 1$ 都成立。因此， $f(n)$ 的增长速度不超过 $g(n)$ 的增长速度。
3. $f(n) = n^3 + n \log n$, $g(n) = n^3$ 。我们有 $f(n) = O(g(n))$ ，因为存在一个常数 $c = 2$ ，使得 $n^3 + n \log n \leq 2n^3$ 对于所有 $n \geq 1$ 都成立。因此， $f(n)$ 的增长速度不超过 $g(n)$ 的增长速度。

大 O 表示法给出了算法运行时间的上界，估的阶越低，则评估精度越高。

大 Ω 表示法 对于函数 $f(n)$ 和 $g(n)$ ，如果存在常数 C 和 n_0 ，使得 $f(n) \geq C \cdot g(n)$ 对于所有 $n \geq n_0$ 都成立，则记作 $f(n) = \Omega(g(n))$ 。也就是说，函数 $f(n)$ 的增长速度不低于 $g(n)$ 。并称 $f(n)$ 的阶不低于 $g(n)$ 的阶。则 $a \geq b$ 。

大 Ω 表示法给出了算法运行时间的下界，估的阶越高，则评估精度越高。

大 Θ 表示法 对于函数 $f(n)$ 和 $g(n)$ ，如果存在常数 C_1, C_2 和 n_0 ，使得 $C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n)$ 对于所有 $n \geq n_0$ 都成立，则记作 $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ 且 $g(n) = O(f(n))$ 。也就是说，函数 $f(n)$ 的增长速度与 $g(n)$ 相同，称 $f(n)$ 和 $g(n)$ 是同阶。则 $a = b$ 。

大 Θ 表示法给出了算法运行时间的确界，理想情况下，希望能够使用大 Θ 表示法表示算法的时间复杂度，退而求其次，使用 Ω 和 O

小 o 表示法 对于函数 $f(n)$ 和 $g(n)$ ，如果对于任意常数 $C > 0$ ，存在常数 n_0 ，使得 $f(n) < C \cdot g(n)$ 对于所有 $n \geq n_0$ 都成立，则记作 $f(n) = o(g(n))$ 。也就是说，函数 $f(n)$ 的增长速度比 $g(n)$ 快。则 $a > b$

类同大 Ω 表示法，但给出的是算法运行时间的上确界

需要注意的是，大 O 表示法、大 Ω 表示法、大 Θ 表示法和小 o 表示法都是在渐进意义下使用的，即表示的是函数在无穷趋势下的增长情况。可以将它们看作是在一个无限大的输入规模下对算法复杂度进行估计的方法。

题 1 ▶ 证明： $n! = o(n^n)$

证明. 对任意给定的 $\varepsilon > 0$, 存在正整数 $n_0 = \lceil \frac{1}{\varepsilon} \rceil \geq \frac{1}{\varepsilon} > 0$, 当 $n \geq n_0 = \lceil \frac{1}{\varepsilon} \rceil$ 时, 有以下不等式成立

$$0 < \frac{n!}{n^n} = \frac{n(n-1)(n-2)\cdots 1}{nn \cdot n} < \frac{nnn\cdots 1}{nn \cdots n} \leq \frac{1}{n} \leq \varepsilon$$

则 $n! = o(n^n)$

综上, $n! = \Theta(n^n)$

□

题 2 ▶ 证明: $\log n! = \Theta(n \log n)$

证明. 由于 $\log n! < \log n^n$, 则有 $\log n! = O(n \log n)$

$$\log n! = \sum_{i=1}^n \log i = \log 1 + \log 2 + \cdots + \log \left(\frac{n}{2}\right) + \cdots + \log n > 0 + \sum_{i=1}^{\lfloor n/2 \rfloor} \log(n/2) = \lfloor n/2 \rfloor \log(n/2) = \lfloor n/2 \rfloor \log n - \lfloor n/2 \rfloor = \Omega(n \log n)$$

综上, $\log n! = \Theta(\log n^n)$

□

题 3 ▶ 证明: $n \sum_{i=1}^n \frac{1}{i} = \Theta(n \log n)$

$$\text{证明. 由于 } \sum_{i=1}^n \frac{1}{i} = 1 + \sum_{i=2}^n \frac{1}{i} \leq 1 + \int_1^n \frac{1}{x} dx = 1 + \ln n$$

$$\sum_{i=1}^n \frac{1}{i} \geq \int_1^{n+1} \frac{1}{x} dx = \ln(n+1)$$

$$\text{因此 } \ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \ln n$$

$$\text{由换底公式可以得到 } \frac{\log(n+1)}{\log e} \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \frac{\log n}{\log e}$$

$$\text{所以 } \sum_{i=1}^n \frac{1}{i} = O(1 + \frac{\log n}{\log e}) = O(\log n)$$

$$\sum_{i=1}^n = \Omega(\frac{\log(n+1)}{\log e}) = \Omega(\log n)$$

$$\text{也就是说 } n \sum_{i=1}^n \frac{1}{i} = \Theta(n \log n)$$

□

题 4 ▶ 证明: $\sum_{i=1}^n \lfloor n/i \rfloor = \Theta(n \log n)$

$$\text{证明. 由 } \sum_{i=1}^n (n/i - 1) \leq \sum_{i=1}^n \lfloor n/i \rfloor \leq \sum_{i=1}^n (n/i)$$

$$\text{则 } \sum_{i=1}^n \lfloor n/i \rfloor = O(\sum_{i=1}^n (n/i)) \text{ 且 } \sum_{i=1}^n \lfloor n/i \rfloor = \Omega(\sum_{i=1}^n (n/i))$$

因此

$$\sum_{i=1}^n \lfloor n/i \rfloor = \Theta\left(\sum_{i=1}^n (n/i)\right)$$

证明 $n \sum_{i=1}^n \frac{1}{i} = \Theta(n \log n)$ (见上)

综上可得 $\sum_{i=1}^n \lfloor n/i \rfloor = \Theta(n \log n)$

□

题 5 ▶ 某算法在解决一个规模为 n 的问题的时候，其所需要的时间可以表示为 $T(n) = \sum_{i=0}^{k-1} (k-i)2^i$ ，其中 $k = \log_2 n$

试用阶的符号来对 $T(n)$ 进行表示

令 $j = k - i$ ，那么 $i = k - j$ ，则 $i = 0 \rightarrow i = k - 1 \Leftrightarrow j = 1 \rightarrow j = k$

因此， $T(n) = \sum_{j=1}^k j2^{k-j} = 2^k \sum_{j=1}^k j2^{-j}$

$$\begin{aligned} \sum_{j=1}^k j2^{-j} &= 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{2^2} + 3 \cdot \frac{1}{2^3} + \cdots + k \cdot \frac{1}{2^k} \\ &= \left(\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \left(\frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \left(\frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \cdots + \frac{1}{2^k} \\ &= \left(1 - \frac{1}{2^k}\right) + \left(\frac{1}{2} - \frac{1}{2^k}\right) + \left(\frac{1}{2^2} - \frac{1}{2^k}\right) + \left(\frac{1}{2^3} - \frac{1}{2^k}\right) + \left(\frac{1}{2^{k-1}} - \frac{1}{2^k}\right) \text{ (看成是公比为 } 2 \text{ 的等比数列求和)} \\ &= \left(\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^{k-1}}\right) + k \cdot \frac{1}{2^k} \\ &= 2\left(1 - \frac{1}{2^k}\right) + k \cdot \frac{1}{2^k} \end{aligned}$$

因此 $T(n) = 2^k \sum_{j=1}^k j2^{-j} = 2^{k+1} - 2 + k = 2n - 2 + \log_2 n = \Theta(n)$

1.4 时间复杂度分析

对于算法中的时间复杂度分析，有两种提问方式，我们采取不同的解决策略：

- 给定一个算法，求解其时间复杂度
 1. 求 $T(n)$ 的上界和下界，分别用 O 和 Ω 进行表示
 2. 上界在**最坏**情形下求出，下界在最理想情形下求出。
 3. 如果上界和下界一致，就可以用 Θ 表示
- 给定一个算法，求解其在最坏情形下的时间复杂度
 1. 求 $T(n)$ 在最坏情形下的上界和下界，分别用 O 和 Ω 进行表示
 2. 上界和下界均在**最坏**情形下求出
 3. 如果上界和下界一致，就可以用 Θ 表示

1.4.1 计算迭代次数

通常，算法耗费的时间和**算法的迭代次数**成比例。因此计算算法的迭代次数就可以作为算法运行时间的指示器。这在很多算法中都可以得到应用，如**查找、排序**等等。

题 1 ▶ 考虑如下所示的算法：

1. 执行该算法，元素比较的最小次数是多少？什么时候达到最小值？
2. 执行该算法，元素比较的最多次数是多少？什么时候达到最大值？
3. 执行该算法，元素赋值的最少次数是多少？什么时候达到最小值？
4. 执行该算法，元素赋值的最多次数是多少？什么时候达到最大值？
5. 用 O 或 Ω 表示算法 BUBBLESORT 的运行时间？
6. 可以用 Θ 来表示算法的运行时间吗？请说明理由

Algorithm 1 BUBBLESORT

Require: $A[1 \dots n]$: 待排序的数组

Ensure: 按非降序排序后的数组 $A[1 \dots n]$

```

1:  $n \leftarrow$  数组长度,  $sorted \leftarrow false$ 
2: for  $i \leftarrow 1$  to  $n - 1$  and not  $sorted$  do
3:    $sorted \leftarrow true$ 
4:   for  $j \leftarrow n$  downto  $i + 1$  do
5:     if  $A[j] < A[j - 1]$  then
6:       交换  $A[j]$  和  $A[j + 1]$ 
7:        $sorted \leftarrow false$ 
8:     end if
9:   end for
10: end for
```

解答：

1. $n - 1$ 次，当输入数组已经按照非降序排列时， $sorted$ 会保持 $true$ ，不会进入 for 的下一循环
2. 最多需要比较 $n(n - 1)/2$ 次，当输入数组中的元素按非升序排列，**且数组中没有相同的元素**
3. 最少需要赋值 0 次，当输入数组已经按照非降序排列
4. 最多需要赋值 $3n(n - 1)/2$ 次（每次交换需要进行 3 次赋值操作），当输入数组中的元素按非升序排列，**且数组中没有相同的元素**
5. $T(n) = O(n^2), T(n) = \Omega(n)$
6. 不可以， Θ 是用来表示算法运行时间的确界，而本算法时间复杂度由 $\Omega(n)$ 到 $O(n^2)$ ，所以不能用这一符号表示

题 2 ▶ 给定如下算法：

1. 当 n 为 2 的幂时，第 5 步执行的最大次数是多少？

2. 当 n 为 3 的幂时，第 5 步执行的最大次数是多少？
3. 用 O 表示算法的时间复杂性是怎样的？
4. 用 Ω 表示的算法的时间复杂性是怎样的？
5. O 和 Ω 那个更适合用来表示算法的时间复杂性？

Algorithm 2 COUNT5

```

1:  $count \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $j \leftarrow \lfloor n/2 \rfloor$ 
4:   while  $j \geq 1$  do
5:      $count \leftarrow count + 1$ 
6:     if  $j$  为奇数 then
7:        $j \leftarrow 0$ 
8:     else
9:        $j \leftarrow j/2$ 
10:    end if
11:  end while
12: end for

```

解答：

1. 设 $n = 2^k$ ，在 n 次循环中， $j = 2^{k-1}$ ，在 **while** 循环中，第 5 步会执行 k 次，而 $k = \log n$ ，故第 5 步执行的最大次数为 $\sum_{i=1}^n k = \sum_{i=1}^n \log n = n \log n$ 次。
2. 设 $n = 3^k$ ，在 n 次循环中， $j = \lfloor 3^k/2 \rfloor$ ，由于是求最大次数，则当且仅当 $j = \lfloor 3^k/2 \rfloor = 2^m$ 时，在 **while** 循环中，第 5 步会执行 $m+1$ 次。故第 5 步的最大执行次数为 $\sum_{i=1}^n (m+1) = n(m+1)$
 考虑到 $\lfloor 3^k/2 \rfloor = \lfloor n/2 \rfloor = 2^m$ ，则有 $n/2 \geq 2^m$ ，即 $\log n \geq m+1$ ，而次数为整数，那么 $m+1 \leq \lfloor \log n \rfloor$
 因此 $\sum_{i=1}^n (m+1) = n(m+1) = n \lfloor \log n \rfloor$
 在 **while** 循环中，第 5 步会执行 k 次，而 $k = \log n$ ，故第 5 步执行的最大次数为 $\sum_{i=1}^n k = \sum_{i=1}^n \log n = n \log n$ 次。
3. 根据前 2 问的分析，其运行时间可用 $O(n \log n)$ 表示
4. 考虑 j 为奇数的情况，这样第 5 步执行次数最小，每次 **while** 循环中只会执行 1 次，总共执行 n 次，因而算法时间复杂度可以用 $\Omega(n)$ 来表示
5. O 更适合表示算法的时间复杂度，该算法不能用 Θ 表示法表示

1.4.2 递归方程

递归方程是描述递归算法时间复杂度的一种常见方式，通常用于描述分治算法的运行时间

它通常是一个关于问题规模 n 的方程，表示算法在处理规模为 n 的问题时所需的时间。递归方程通常包括两部分：一个基本情况（通常是 $n = 1$ 或 $n = 0$ 时的情况），以及一个或多个递归情况，描述了问题规模减小时算法的行为。

以经典的分治算法——归并排序为例，来看一下如何计算递归方程的时间复杂度。

归并排序的递归方程可以描述为：

$$T(n) = 2T(n/2) + n$$

这里， n 是待排序数组的长度。归并排序的工作原理是将数组分为两半，分别排序，然后合并两个已排序的子数组。因此， $T(n) = 2T(n/2) + n$ ，其中 $2T(n/2)$ 代表分别对两个子数组进行排序的时间， n 代表合并两个子数组的时间。

我们可以通过主定理（Master Theorem）来求解这种递归方程的时间复杂度。主定理提供了一种方法，可以直接得出形如 $T(n) = aT(n/b) + f(n)$ 这类递归方程的解。

对于归并排序的递归方程， $a = 2, b = 2, f(n) = n$ 。根据主定理，归并排序的时间复杂度为 $O(n \log n)$ 。在第二章中涉及到这个方法。

1.4.3 频度分析

在后续分析 Dijkstra 算法的时间复杂度的过程中，我们应用了频度分析。

1.4.4 复杂度分析的意义

算法复杂度分析是计算机科学中的一种重要技术，它为我们提供了一种量化算法性能的手段。具体来说，算法复杂度分析可以帮助我们理解算法执行所需的资源。这些资源通常包括两种：时间和空间。

对于时间复杂度，我们通常关心随着输入大小的增长，算法执行时间的增长趋势。这被称为算法的时间复杂度。例如，一个算法可能具有线性时间复杂度 $O(n)$ ，这意味着随着输入大小的增长，其执行时间也会成比例地增长。

空间复杂度分析则关心算法执行过程中所需的额外空间。例如，一个具有线性空间复杂度 $O(n)$ 的算法将随着输入大小的增加，需要的额外空间也会成比例地增长。

为什么我们需要关心算法复杂度呢？有几个原因：

- 性能预测：通过了解算法的复杂度，我们可以预测算法在处理大量数据时的表现。这对于评估算法的可扩展性至关重要。
- 算法选择：当我们有多种解决同一问题的算法时，复杂度分析可以帮助我们选择最有效的算法。
- 优化：通过理解算法的复杂度，我们可以找出优化算法性能的机会。例如，如果我们看到一个算法有较高的时间复杂度，我们可能会寻找减少运行时间的方法。

总的来说，算法复杂度分析是评估和优化算法性能的关键工具。

2 数学基础与数据结构

2.1 递归方程求解

2.1.1 常系数线性同质递归方程

对于给定的常数 $a_1, a_2, \dots, a_k$ 和一个初始条件，一个 k 阶常系数线性同质递归方程可以表示为：

$$T(n) = a_1 T(n-1) + a_2 T(n-2) + \dots + a_k T(n-k)$$

对于适当的初始条件，例如 $T(0), T(1), \dots, T(k-1)$ 。

其特征方程为

$$x^k = a_1 x^{k-1} + a_2 x^{k-2} + \dots + a_k$$

我们此处仅仅关注一阶和二阶同质方程。

对于一阶情况，求解过程是递推的：

$$T(n) = aT(n-1) = a^2 T(n-2) = \dots = a^n T(0)$$

对于二阶情况，特征方程为：

$$x^2 - a_1 x - a_2 = 0$$

其中， a_1 为 $T(n-1)$ 的系数， a_2 为 $T(n-2)$ 的系数。

设该二次方程的两个根为 x_1, x_2 。那么 $T(n)$ 可以表示为：

$$T(n) = \begin{cases} c_1 x_1^n + c_2 x_2^n, & x_1 \neq x_2 \quad \& \quad x_1, x_2 \in \mathbf{R} \\ (c_1 n + c_2) r^n, & x_1 = x_2 = r \in \mathbf{R} \\ r^n (c_1 \cos(n\theta) + c_2 \sin(n\theta)), & x_1, x_2 \notin \mathbf{R} \end{cases}$$

然后利用初始值 $T(n_0)$ 和 $T(n_0+1)$ 求解即可。

例题 1 ▶ 已知 $T(n) = 3T(n-1) + 4T(n-2)$ ，并且 $T(0) = 1, T(1) = 4$

根据特征方程的形式 $x^2 - a_1 x - a_2 = 0$ 写出该题的特征方程 $x^2 - 3x - 4 = 0$ ，由此可以解得特征根 $x_1 = -1, x_2 = 4$

根据解的情况进行讨论，此处属于情形 $x_1 \neq x_2 \in \mathbf{R}$ 的情况，故可以写出 $T(n) = c_1(-1)^n + c_2 4^n$

现在需要根据初始值来求解，根据已知条件：

$$\begin{cases} T(0) = c_1 + c_2 = 1 \\ T(1) = -c_1 + 4c_2 = 4 \end{cases}$$

解得 $c_1 = 0, c_2 = 1$ ，从而 $T(n) = 4^n = \Theta(4^n)$

例题 2 ▶ 已知 $f(n) = \begin{cases} -6f(n-1) - 9f(n-2) & n \geq 2 \\ -3 & n = 1 \\ 3 & n = 0 \end{cases}$

试给出 $f(n)$ 的解析式。

特征方程 $x^2 + 6x + 9 = 0$ ，解得特征根 $x_1 = x_2 = -3$ 。

根据解得情况进行讨论，此处属于情形 $x_1 = x_2 \in R$ 的情形，故可以写出 $f(n) = (c_1n + c_2)(-3)^n$

根据已知条件有：
$$\begin{cases} T(0) = c_2 = 3 \\ T(1) = -3(c_1 + c_2) = -3 \end{cases}$$

解得 $c_1 = -2, c_2 = 3$

故 $f(n) = (-2n + 3)(-3)^n$

例题 3 ▶ 给定一个规模为 n 的问题实例。假设存在分治算法 A 和 B 可以用来解决该问题。

算法 A：将规模为 n 的问题分解为 5 个规模为 $n-1$ 和 6 个规模为 $n-2$ 的子问题；然后，将所有 11 个子问题的解进行合并，得到原问题的解。

算法 B：将规模为 n 的问题分解为 2 个规模为 $n-1$ 和 8 个规模为 $n-2$ 的子问题；然后，将所有 10 个子问题的解进行合并，得到原问题的解。

假设，上述两个算法中合并子问题的解均不需要额外的时间。

问：从时间复杂度的角度来讲，算法 A 和 B 哪个较优？

算法 A：由题目描述可知， $T(n) = 5T(n-1) + 6T(n-2)$

其特征方程为 $x^2 - 5x - 6 = 0$ ，解得特征根 $x_1 = 6, x_2 = -1$ ，则 $T(n) = c_16^n + c_2(-1)^n$

算法 B：由题目描述可知， $T(n) = 2T(n-1) + 8T(n-2)$

其特征方程为 $x^2 - 2x - 8 = 0$ ，解得特征根 $x_1 = 2, x_2 = -4$ ，则 $T(n) = c_12^n + c_2(-4)^n$

从时间复杂度的角度来看，算法 B 较优。

2.1.2 几类特殊的非同质递归方程求解

- 第一类： $T(n) = T(n-1) + g(n), n \geq 1$

求解方法：

$$T(n) = T(0) + \sum_{i=1}^n g(i)$$

- 第二类： $T(n) = g(n)T(n-1), n \geq 1$
- 第三类： $T(n) = g(n)T(n-1) + h(n), n \geq 1$

$$\text{令 } k(n) = \begin{cases} \frac{T(n)}{g(n)g(n-1)\cdots g(1)}, & n \geq 1 \\ T(0), & n = 0 \end{cases}$$

则 $T(n) = g(n)g(n-1)\cdots g(1)k(n) (n \geq 1)$

则 $T(n-1) = g(n-1)\cdots g(1)k(n-1)$

而 $T(n) = g(n)T(n-1) + h(n)$

则 $g(n)g(n-1)\cdots g(1)k(n) = g(n)g(n-1)\cdots g(1)k(n-1) + h(n)$

两边同时除以 $g(n)g(n-1)\cdots g(1)$ 可以得到 $k(n) = k(n-1) + \frac{h(n)}{g(n)\cdots g(1)}$ ，回到情形 1

$$\text{则 } k(n) = k(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)} = T(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)}$$

对于 $T(n) = g(n)g(n-1)\cdots g(1)k(n) (n \geq 1)$

$$T(n) = \begin{cases} g(n)g(n-1)\cdots g(1)(T(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)}) & n \geq 1 \\ T(0) & n = 0 \end{cases}$$

例题 1 ▶ 设 $T(n) = nT(n-1) + n!, T(0) = 0$

上式与情形 3 相契合，其中 $g(n) = n, h(n) = n!$

直接代入公式有 $T(n) = n!(0 + \sum_{i=1}^n \frac{i!}{i!}) = n!n$

例题 2 ▶ 对于一个规模为 n 的数组，快速排序的时间复杂度可以用下式递归表示：

$$T(n) = (n-1) + \frac{1}{n} \sum_{w=1}^n (T(w-1) + T(n-w))$$

试用同阶符号 Θ 来表示 $T(n)$

$$\text{注意到 } \sum_{w=1}^n T(n-w) = T(0) + T(1) + T(2) + \cdots + T(n-1) = \sum_{w=1}^n T(w-1)$$

$$\text{因此 } T(n) = (n-1) + \frac{2}{n} \sum_{w=1}^n T(w-1)$$

$$\text{将之作代数变形有 } nT(n) = n(n-1) + 2 \sum_{w=1}^n T(w-1)$$

$$\text{用 } n-1 \text{ 迭代 } n \text{ 有 } (n-1)T(n-1) = (n-1)(n-2) + 2 \sum_{w=1}^{n-1} (n-1)T(w-1)$$

$$\text{两式相减有 } nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1)$$

$$\text{从而 } nT(n) = (n+1)T(n-1) + 2(n-1)$$

$$\text{代数变形，同时除以 } n(n+1)，\text{ 有 } \frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

$$\text{事实上，可以令 } a(n) = \frac{T(n)}{n+1}，\text{ 则上式转换为 } a(n) = a(n-1) + \frac{2(n-1)}{n(n+1)}$$

$$\text{对 } \frac{n-1}{n(n+1)} \text{ 进行裂项（采用待定系数法），可以得到 } \frac{n-1}{n(n+1)} = \frac{2}{n+1} - \frac{1}{n}$$

那么由第一类特殊的非同质递归方程的求解方法可以得到

$$a(n) = a(0) + 2 \sum_{i=1}^n \frac{i-1}{i(i+1)} = 4 \sum_{i=2}^{n+1} \frac{1}{i} - 2 \sum_{i=1}^n \frac{1}{i} = 2 \sum_{i=1}^n \frac{1}{i} - \frac{4n}{n+1} = \Theta(\log n)$$

$$T(n) = (n+1)a(n) = \Theta(n \log n)$$

2.1.3 主定理 (Master Theorem)

主定理适用于以下形式的递归方程：

$$T(n) = aT(n/b) + f(n)$$

其中, $a \geq 1, b > 1$, $f(n)$ 是一个正的函数。这个公式表示将问题的规模缩小到 $1/b$, 产生 a 个子问题, 并且额外的工作由 $f(n)$ 给出, 并且 $T(n)$ 有适当的初始值。

1. 如果 $f(n) = O(n^{\log_b a - \varepsilon})$, 其中 $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$ 。

注： ε 是一个正的小量, 用于界定 $f(n)$ 的增长速度。

$f(n) = O(n^{\log_b a - \varepsilon})$, 说明 $f(n)$ 的增长速度小于 $n^{\log_b a}$

2. 如果 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

$f(n) = \Theta(n^{\log_b a})$, 说明 $f(n)$ 的增长速度与 $n^{\log_b a}$ 相当。

3. 如果 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 其中 $\varepsilon > 0$, 如果对某个常数 $c < 1$, 有 $af(n/b) \leq cf(n)$ 对所有充分大的 n 都成立, 那么 $T(n) = \Theta(f(n))$ 。

$f(n) = \Omega(n^{\log_b a + \varepsilon})$, 说明 $f(n)$ 的增长速度大于 $n^{\log_b a}$ 。

例题 1 ▶ 设 $T(n) = T(n/2) + 1$, 求 $T(n)$ 的时间复杂度

此处, $a = 1, b = 2$, 则 $\log_b a = \log_2 1 = 0$

$s(n) = n^0 = 1$, 与 $f(n) = 1$ 同阶, 因此适用主定理的第二种情况, 故 $T(n) = \Theta(\log n)$

例题 2 ▶ 设 $T(n) = 9T(n/3) + (n - 1)$, 求 $T(n)$ 的时间复杂度

此处, $a = 9, b = 3$, 则 $\log_b a = \log_3 9 = 2$

$s(n) = n^2, f(n) = n - 1$, $f(n)$ 的增长速度小于 $s(n)$, 适用于主定理的第一种情况, 故 $T(n) = \Theta(n^2)$

例题 3 ▶ 设 $T(n) = 3T(n/4) + n \log n$

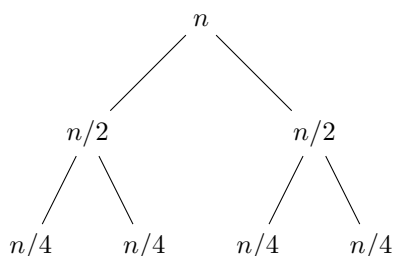
此处, $a = 3, b = 4, \log_b a \approx 0.793$

则 $s(n) \approx n^{0.79}$, 由于 $f(n) = n \log n$ 的增长速度快于 $s(n)$, 适用于主定理的第三种情况, 因此 $T(n) = \Theta(n \log n)$

2.1.4 递归树

递归树 (Recursion Tree) 是一种图形化的方法, 用于求解递归方程。在递归树中, 每个节点都表示递归方程的一次调用, 节点的值表示该次调用的成本。树的总成本就是所有节点的成本之和, 这就是递归方程的解。

例如, 对于递归方程 $T(n) = 2T(n/2) + n$, 我们可以画出如下的递归树:



然后，我们可以将每一层的成本相加，最后得到总成本，这就是递归方程的解。

2.1.5 代入法

代入法 (Substitution Method) 是一种试探性的方法，用于求解递归方程，我们通过一道题来说明代入法。

题 1 ▶ 设 a 和 c 是非负整数， b, d, x 是非负整数，并且对于某个非负整数 k ，有 $n = c^k$ ，那么某个算法的时间复杂度 $f(n)$ 满足以下递推方程

$$f(n) = \begin{cases} d & n = 1 \\ af(n/c) + bn^x & \end{cases}$$

证明：当满足 $a = c^x$ 的条件下，时间复杂度 $f(n)$ 的阶等于 $\Theta(n^x \log n)$

证明. 对 $f(n)$ 递推公式进行展开：

$$\begin{aligned}
 f(n) &= af(n/c) + bn^x \\
 &= a(af(n/c^2) + b(n/c)^x) + bn^x \\
 &= a^k f(n/c^k) + bn^x \sum_{i=0}^{k-1} (a/c^x)^i \\
 &= a^k f(1) + bn^x \sum_{i=0}^{k-1} (a/c^x)^i \\
 &= a^k d + bn^x \sum_{i=0}^{k-1} 1^i \\
 &= a^k d + bn^x k \\
 &= a^{\log_c n} d + bn^x \log_c n \\
 &= n^{\log_c a} d + bn^x \log n / \log c \\
 &= dn^x + bn^x \log n / \log c
 \end{aligned}$$

显然，该多项式的第二项式更高，因此有 $f(n) = \Theta(n^x \log n)$

□

2.2 数据结构

2.2.1 堆

在许多算法中，需要大量用到如下两种基本操作：**插入元素**和**寻找最大（小）值元素**。为了提高这两种运算的效率，必须使用恰当的数据结构。堆是一种有效实现上述两种运算的数据结构。

堆是一个几乎完全的二叉树，每个节点都满足这样的特性：**任一父节点的键值 (key) 不小于 (大于等于) 子节点的键值**。(这里以最大堆为例，最小堆将不小于改为不大于)

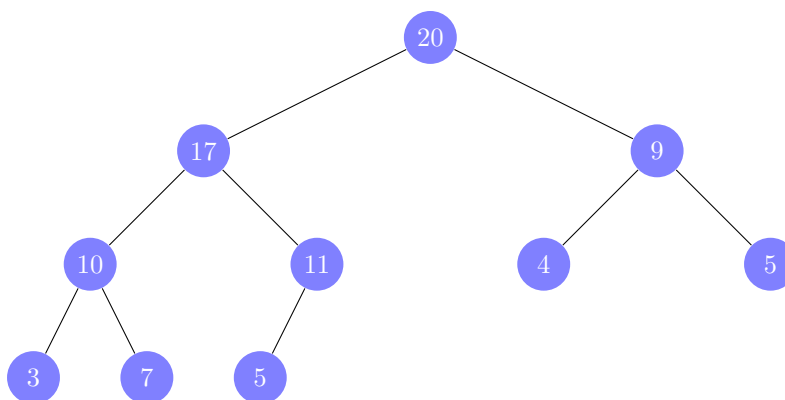
有 n 个节点的堆 T ，可以用一个数组 $H[1...n]$ 来表示：

- T 的**根节点**存储在 $H[1]$ 中
- 假设 T 的节点 x 存储在 $H[i]$ 中，那么，它的**左右孩子节点**分别存放在 $H[2i]$ 及 $H[2i + 1]$ 中。
- $H[i]$ 的父节点如果不是根节点，则其**父节点**存储在 $H[\lfloor i/2 \rfloor]$ 中

一个满足堆性质的数组如下所示：

20	17	9	10	11	4	5	3	7	5
----	----	---	----	----	---	---	---	---	---

转换成树形结构也即：



以最大堆为例介绍堆的性质：

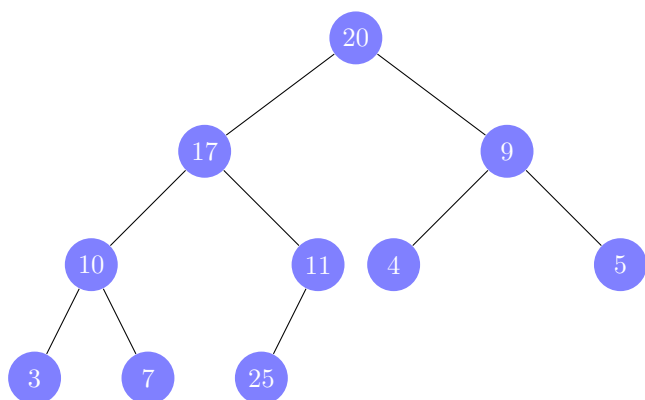
- **根节点键值最大**，叶子节点键值较小。从根到叶子，键值以**非升序**排列。
- 节点的左右孩子节点键值**并无顺序要求**。
- 堆的数组表示呈“基本有序”状态（所以，堆又称为**优先队列**）。此外，并非节点的高度越高，键值就越大。

最大堆的基本操作：

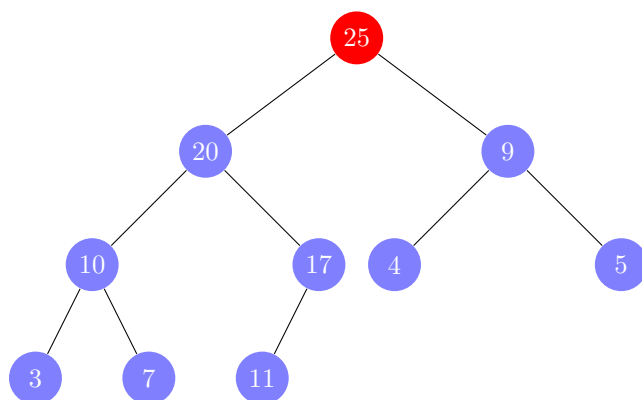
- `insert(H, x)`: 插入元素 x 到堆 H 中

辅助运算 Sift-up (以最大堆为例) $O(\log n)$

- 若某个节点 $H[i]$ 的键值大于其父节点的键值，就违背了最大堆的特性，需要进行调整。
- 调整方法：**上移**。
- 沿着 $H[i]$ 到根节点的**唯一一条路径**，将 $H[i]$ 移动到合适的位置上：比较 $H[i]$ 及其父节点 $H[\lfloor i/2 \rfloor]$ 的键值，若 $key(H[i]) > key(H[\lfloor i/2 \rfloor])$ ，则二者进行交换，直到 $H[i]$ 到达合适位置。



(a) 节点 25 有误的堆



(b) 经过 Sift-up 操作恢复后的堆

Algorithm 3 Sift-up(H, i)

Require: 数组 $H[1..n]$, 索引 i ($1 \leq i \leq n$)

Ensure: 上移 $H[i]$ (如果需要), 使其键值不大于父节点的键值

```

1:  $done \leftarrow false$ 
2: if  $i = 1$  then
3:   exit {根节点, 无需上移, 直接退出}
4: end if
5: repeat
6:   if  $key(H[i]) > key(H[\lfloor i/2 \rfloor])$  then
7:     互换  $H[i]$  和  $H[\lfloor i/2 \rfloor]$ 
8:   else
9:      $done \leftarrow true$  {调整过程已满足要求, 退出}
10:  end if
11:   $i \leftarrow \lfloor i/2 \rfloor$ 
12: until  $i = 1$  or  $done$  {调整进行到根节点或到某一节点终止}
    
```

辅助运算 Sift-down(H, i) $O(\log n)$

- 假设某个内部节点 $H[i]$ ($i \leq \lfloor \frac{n}{2} \rfloor$), 其键值小于其孩子节点的键值, 即 $key(H[i]) < key(H[2i])$ 或 $key(H[i]) < key(H[2i+1])$ (如果右孩子存在)。这违背了堆的特性, 需要进行调整。
- 调整方法: 下渗。
- 沿着从 $H[i]$ 到孩子节点 (可能不唯一, 取孩子节点中键值较大者) 的路径, 比较 $H[i]$ 与孩子节点的键值, 若 $key(H[i]) < \max(key(H[2i]), key(H[2i+1]))$, 则交换它们。这一过程一直进行, 直到节点成为叶子节点或满足堆的特性为止。

Algorithm 4 Sift-up(H, i)

Require: 数组 $H[1..n]$, 索引 i ($1 \leq i \leq n$)

Ensure: 上移 $H[i]$ (如果需要), 使其键值不大于父节点的键值

```

1:  $done \leftarrow false$ 
2: if  $i = 1$  then
3:   exit {根节点, 无需上移, 直接退出}
4: end if
5: repeat
6:   if  $key(H[i]) > key(H[\lfloor i/2 \rfloor])$  then
7:     互换  $H[i]$  和  $H[\lfloor i/2 \rfloor]$ 
8:   else
9:      $done \leftarrow true$  {调整过程已满足要求, 退出}
10:  end if
11:   $i \leftarrow \lfloor i/2 \rfloor$ 
12: until  $i = 1$  or  $done$  {调整进行到根节点或到某一节点终止}
```

插入操作的思路: 先将 x 添加到 H 的末尾, 然后利用 Sift-up, 调整 x 在 H 中的位置, 直到满足堆特性。

Algorithm 5 Insert(H, x)

Require: 堆 $H[1..n]$ 和元素 x

Ensure: 新堆 $H[1..n+1]$, x 是其中元素之一

```

1:  $n \leftarrow n + 1$  {堆大小增 1}
2:  $H[n] \leftarrow x$ 
3: SIFT-UP( $H, n$ ) {调整堆}
```

树的高度为 $\lceil \log n \rceil$, 所以将一个元素插入大小为 n 的堆所需要的时间是 $O(\log n)$ 。

- **delete**(H, i): 删除堆 H 的第 i 项

先用 $H[n]$ 取代 $H[i]$, 然后对 $H[i]$ 作 Sift-up 或 Sift-down, 直到满足堆特性。

Algorithm 6 Delete(H, i)

Require: 非空堆 $H[1...n]$, 索引 i , $1 \leq i \leq n$

Ensure: 删除 $H[i]$ 后的新堆 $H[1...n-1]$

```

1:  $x \leftarrow H[i], y \leftarrow H[n]$ 
2:  $n \leftarrow n - 1$  {堆大小减 1}
3: if  $i = n + 1$  then
4:   exit {要删除的刚好是最后一个元素，叶节点}
5: end if
6:  $H[i] \leftarrow y$  {用原来的  $H[n]$  取代  $H[i]$ }
7: if  $\text{key}(y) \geq \text{key}(x)$  then
8:   SIFT-UP( $H, i$ )
9: else
10:  SIFT-DOWN( $H, i$ )
11: end if

```

类似地，容易知道删除操作的时间复杂度为 $O(\log n)$

- **deleteMax(H):** 从非空堆 H 中删除最大键值并返回数据项 max

Algorithm 7 deleteMax(H)

Require: 堆 $H[1...n]$

Ensure: 返回最大键值元素，并将其从堆中删除

```

1:  $x \leftarrow H[1]$ 
2: DELETE( $H, 1$ ) {调用删除算法删除堆顶元素}
3: return  $x$ 

```

- **make_heap(A):** 从数组 A 创建堆 H

方法一：从一个空 H 堆开始，逐步插入 A 中的每个元素，直到 A 中所有元素都被转移到堆 H 中。(调用 **insert(H, x)**)

时间复杂度： $O(n \log n)$

方法二：从最后一个非叶子节点开始，逐个调用“下移”操作（SIFT-DOWN）来调整子树，以确保每个节点都满足堆的性质。这个过程从底部向上进行，逐步处理每个非叶子节点，直到根节点。

Algorithm 8 MakeHeap(A)

Require: 数组 A

Ensure: 将数组 A 转换成堆

```

1: for  $i \leftarrow \lfloor \frac{n}{2} \rfloor$  downto 1 do
2:   SIFT-DOWN( $A, i$ ) {以  $A[i]$  为根节点的子树调整成堆，调用 SIFT-DOWN 过程}
3: end for

```

树高 $k = \lceil \log n \rceil$ ，对于第 i 层，有 2^i 个节点，Sift-down 操作只对 $0 \leq i < k$ 进行，即不含最深的叶子结点

层，每个节点的 Sift-down 操作至多执行 $k-i$ 次，也即下沉到叶子结点层，故 down 过程执行次数上限为：

$$\sum_{i=0}^{k-1} (k-i)2^i$$

令 $j = k - i$ ，那么 $i = k - j$ ，则 $i = 0 \rightarrow i = k - 1 \Leftrightarrow j = 1 \rightarrow j = k$

$$\text{因此，} T(n) = \sum_{j=1}^k j2^{k-j} = 2^k \sum_{j=1}^k j2^{-j}$$

$$\begin{aligned} \sum_{j=1}^k j2^{-j} &= 1\frac{1}{2} + 2\frac{1}{2^2} + 3\frac{1}{2^3} + \cdots + k\frac{1}{2^k} \\ &= \left(\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \left(\frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \left(\frac{1}{2^3} + \cdots + \frac{1}{2^k}\right) + \cdots + \frac{1}{2^k} \\ &= \left(1 - \frac{1}{2^k}\right) + \left(\frac{1}{2} - \frac{1}{2^k}\right) + \left(\frac{1}{2^2} - \frac{1}{2^k}\right) + \left(\frac{1}{2^3} - \frac{1}{2^k}\right) + \left(\frac{1}{2^{k-1}} - \frac{1}{2^k}\right) \text{ (看成是公比为 } 2 \text{ 的等比数列求和)} \\ &= \left(\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^{k-1}}\right) + k\frac{1}{2^k} \\ &= 2\left(1 - \frac{1}{2^k}\right) + k\frac{1}{2^k} \end{aligned}$$

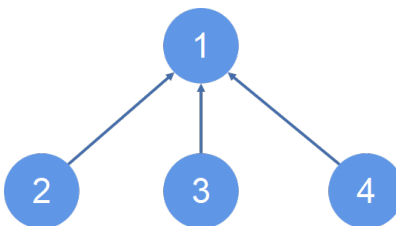
$$\text{因此 } T(n) = 2^k \sum_{j=1}^k j2^{-j} = 2^{k+1} - 2 + k = 2n - 2 + \log_2 n = \Theta(n) \text{ 时间复杂度为 } \Theta(n)$$

2.2.2 并查集（不相交集）

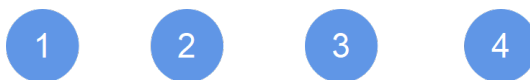
并查集 (Disjoint Sets)，也称为不相交集，用于管理元素的分组和合并。它主要支持两个操作：**查找** (Find) 和 **合并** (Union)。

在并查集中，每个元素都被分配一个唯一的标识符，通常是一个整数。这些元素可以被划分为若干个不相交的集合，每个集合都有一个代表元素，也称为根元素或根节点。**每个元素都指向其所属集合的根节点。**

在具体实现过程中，每个集合可以用树表示，除根节点外，每个节点都有指针指向父节点。用根节点表示集合的名字。例如，集合 1: {1,2,3,4} 可以记为如下形式：



初始状态下，每个元素都是一个单独的集合，每个集合的根节点就是元素自身。



查找操作 **FIND(x)** 用于确定元素 x 所属的集合名字。它通过追踪元素指向的根节点来实现。如果两个元素的根节点相同，表示它们属于同一个集合；如果根节点不同，则表示它们属于不同的集合。**FIND(x)** 返回 **root(x)** 的值。

执行合并操作 **UNION(x,y)** 时，包含 x 和包含 y 的根节点被连接在一起，形成一个更大的集合。这样，两个元素就属于同一个集合了。具体来说，执行合并操作 **UNION(x,y)** 时，首先需要使用 **FIND(x)** 获取 **root(x)**，记为 u 。使用 **FIND(y)** 获取 **root(y)**，记为 v 。再令 u 指向 v 即可。

例如，对上面的集合执行 **UNION(1,2)**, **UNION(4,3)** 可以得到：



但是，多次合并后，树的高度可能很大，使得后续执行 FIND 操作的时间变长。
为了降低树的高度，采用秩启发的方式。

1. 给每个树的根节点定义一个秩 (rank)，表示该树的高度（也即祖宗节点到该节点的距离）
2. 执行 UNION(x, y) 时，首先找到 $u = \text{root}(x), v = \text{root}(y)$
3. 比较 $\text{rank}(u)$ 和 $\text{rank}(v)$
 - 若 $\text{rank}(u) = \text{rank}(v)$ ，则使 u 指向 v ， v 成为 u 的父亲，同时 $\text{rank}(v)++$
 - 若 $\text{rank}(u) < \text{rank}(v)$ ，则使 u 指向 v ， v 成为 u 的父亲
 - 若 $\text{rank}(u) > \text{rank}(v)$ ，则使 v 指向 u ， v 称为 v 的父亲

Algorithm 9 UNION

Require: 两个元素 x, y

Ensure: 合并包含 x, y 的两棵树

```

1:  $u \leftarrow \text{FIND}(x)$ 
2:  $v \leftarrow \text{FIND}(y)$ 
3: if  $\text{rank}(u) \leq \text{rank}(v)$  then
4:    $p(u) \leftarrow v$  {将  $v$  设置为  $u$  的父节点，使  $u$  指向  $v$ }
5:   if  $\text{rank}(u) = \text{rank}(v)$  then
6:      $\text{rank}(v) \leftarrow \text{rank}(v) + 1$ 
7:   end if
8: else
9:    $p(v) \leftarrow u$  {将  $u$  设置为  $v$  的父节点，使  $v$  指向  $u$ }
10: end if
    
```

为了进一步提高 FIND 操作的性能，采用路径压缩的方式。

路径压缩是一种在查找操作中优化树的高度的技术。当执行查找操作时，除了找到根节点外，还会通过修改指向父节点的指针，将遍历路径上的每个节点直接连接到根节点，以缩短树的高度。这样可以在以后的查找操作中更快地找到根节点。

Algorithm 10 FIND

Require: 节点 x

Ensure: $root(x)$ 和路径压缩后的树

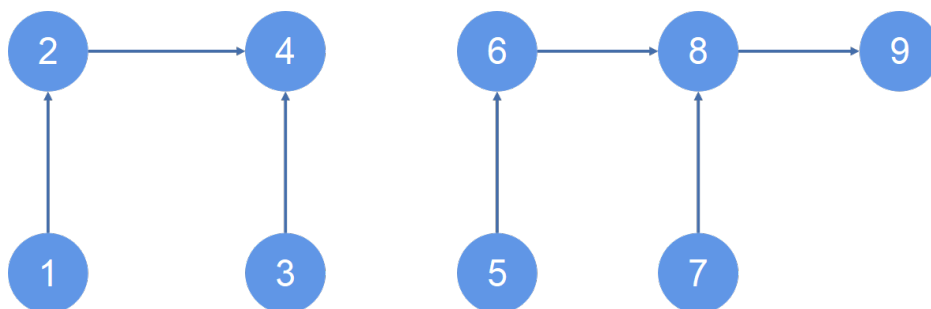
```

1:  $y \leftarrow x$ 
2: while  $p(y) \neq \text{null}$  do
3:    $y \leftarrow p(y)$  {寻找包含  $x$  的树的根}
4: end while
5:  $root \leftarrow y$ ;  $y \leftarrow x$  {重新赋值为原来的节点  $x$ }
6: while  $p(y) \neq \text{null}$  do
7:    $w \leftarrow p(y)$  {执行路径压缩, 父节点暂存为  $w$ }
8:    $p(y) \leftarrow root$  {该路径上的节点直接指向根节点}
9:    $y \leftarrow w$  {继续下一步压缩}
10: end while
11: return  $root$ 

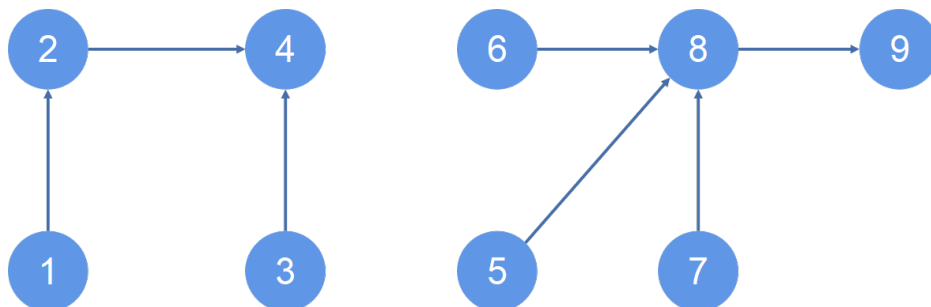
```

注：修改了 FIND 操作后，由于 UNION 操作内含 FIND 的操作，需要注意在执行 UNION 的时候也会存在路径压缩的情况。

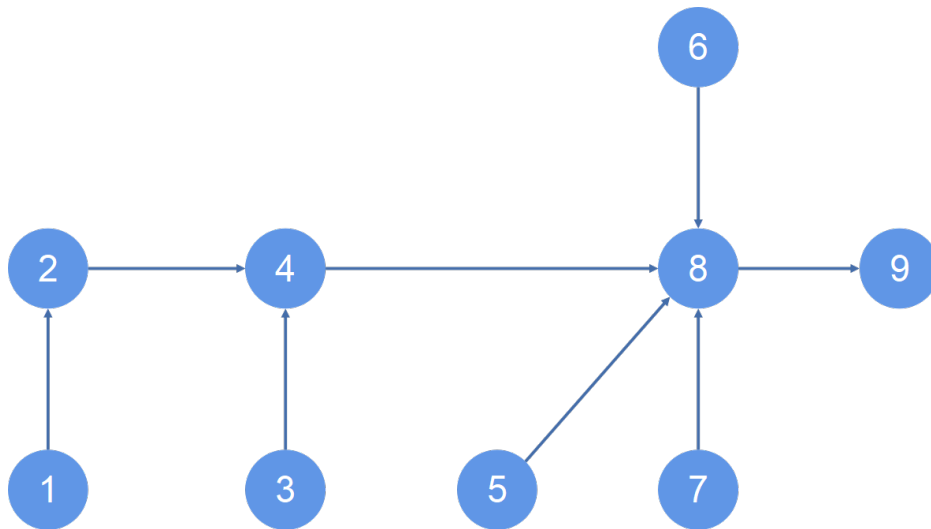
给定并查集如下所示：



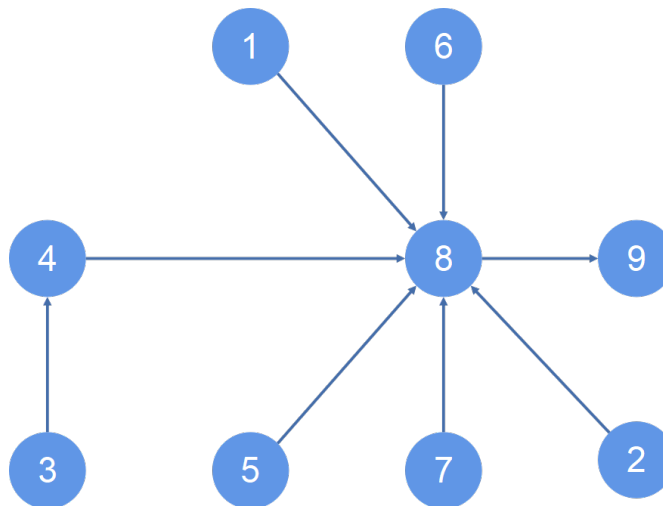
执行 FIND(5) 后得到的结果如下图所示：（5 直接指向根节点）



继续执行 UNION(4, 8) 后得到的结果如下图所示：（4 直接指向 8）



继续执行 FIND(1) 后得到的结果如下图所示：（1 指向 8, 2 也指向 8）



3 分治策略

$$d(i, j, c) = \begin{cases} 0, & A[1], d[i-1, j, C] \max(d[i-1, j-1, C-A[i]], d[i-1, j, C]) \end{cases}$$

3.1 基本思想

分治策略的思想：

- 把规模较大的问题分解为若干个规模较小的子问题，这些子问题相互独立且与原问题同类；（该子问题的规模减小到一定的程度就可以容易地解决）
- 依次求出这些子问题的解，然后把这些子问题的解组合起来得到原问题的解。
- 由于子问题与原问题是同类的，故分治法可以很自然地应用递归。

分治法的适用条件：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的同类问题；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划更为合适。

3.2 矩阵相乘

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

它们的乘积 AB 的形式可以表示为：

$$AB = \begin{bmatrix} \sum_{k=1}^n a_{1k}b_{k1} & \sum_{k=1}^n a_{1k}b_{k2} & \cdots & \sum_{k=1}^n a_{1k}b_{kn} \\ \sum_{k=1}^n a_{2k}b_{k1} & \sum_{k=1}^n a_{2k}b_{k2} & \cdots & \sum_{k=1}^n a_{2k}b_{kn} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^n a_{nk}b_{k1} & \sum_{k=1}^n a_{nk}b_{k2} & \cdots & \sum_{k=1}^n a_{nk}b_{kn} \end{bmatrix}$$

假设每做一次标量乘法耗费的时间为 m ，则在乘法上消耗的时间为 $n^2 \cdot n \cdot m$ （每一个方格内都有 n 次乘法），每做一次标量加法耗费的时间为 a ，则在加法上消耗的时间为 $n^2(n-1)a$ （每一个方格内都有 $n-1$ 次加法）

那么 $T(n) = n^3m + n^3a - n^2a = \Theta(n^3)$

当将矩阵 $A_{n \times n}$ 和 $B_{n \times n}$ 进行分块时，可以将它们划分为 4 个子块。

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

对于乘积 AB ，我们可以使用分块矩阵乘法来表示：

$$AB = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

$$\text{那么 } T(n) = \begin{cases} m & n = 1 \\ 8T(n/2) + 4(n/2)^2a & n \geq 2 \end{cases}$$

由主定理可以得到 $T(n) = \Theta(n^3)$

Strassen 进一步改进了该算法，优化了算法的时间复杂度。

1. 如果矩阵 A 和矩阵 B 的大小都为 1×1 ，即单个元素的矩阵，直接计算它们的乘积并返回结果。
2. 否则，将矩阵 A 和矩阵 B 分别划分成四个子矩阵： A_{11} 、 A_{12} 、 A_{21} 、 A_{22} 和 B_{11} 、 B_{12} 、 B_{21} 、 B_{22} 。
3. 递归地调用 Strassen 算法，对子矩阵进行乘法运算。计算以下七个子问题：

$$P_1 = \text{Strassen}(A_{11} + A_{22}, B_{11} + B_{22})$$

$$P_2 = \text{Strassen}(A_{21} + A_{22}, B_{11})$$

$$P_3 = \text{Strassen}(A_{11}, B_{12} - B_{22})$$

$$P_4 = \text{Strassen}(A_{22}, B_{21} - B_{11})$$

$$P_5 = \text{Strassen}(A_{11} + A_{12}, B_{22})$$

$$P_6 = \text{Strassen}(A_{21} - A_{11}, B_{11} + B_{12})$$

$$P_7 = \text{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})$$

利用这些子问题的解，通过一些加法和减法操作计算出结果矩阵 C 的四个子矩阵：

$$C_{11} = P_1 + P_4 - P_5 + P_7$$

$$C_{12} = P_3 + P_5$$

$$C_{21} = P_2 + P_4$$

$$C_{22} = P_1 - P_2 + P_3 + P_6$$

将这四个子矩阵合并成最终的结果矩阵 C。

$$T(n) = \begin{cases} m & n = 1 \\ 7T(n/2) + 18(n/2)^2 a & n \geq 2 \end{cases}$$

$$\text{因此 } T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

3.3 大整数相乘

通常，在分析算法的计算复杂度时，都将标量加法和标量乘法运算当作基本运算来处理，即将执行一次标量加法或标量乘法运算所需要的计算时间当作一个常数，该常数仅仅取决于计算机硬件处理速度。

然而，这个假定仅仅在参加运算的整数处于一定范围内时才是合理的。这个整数范围取决于计算机硬件对整数的表示。

在某些情况下，要处理很大的整数，它无法在计算机硬件能直接表示的整数范围内进行处理。这时候，就必须使用软件的方法来实现大整数的算术运算。

问题：设有两个 n bit 的整数 X, Y ，要计算 XY

1. 将 X, Y 转换成二进制耗费的时间为 $\Theta(n)$
2. X 与 Y 的二进制位相乘，耗费的时间为 $\Theta(n^2)$
3. 将 XY 转换为十进制，耗费的时间为 $\Theta(n)$

因此，上述算法的时间复杂度为 $\Theta(n^2)$

如果采用分治策略，递归地分割 n 个 bit 的整数 X, Y 分割为 $n/2$ 位的 $X(A, B), Y(C, D)$

则 $X = A \cdot 2^{n/2} + B, Y = C \cdot 2^{n/2} + D$

则 $XY = (A \cdot 2^{n/2} + B)(C \cdot 2^{n/2} + D) = A \cdot C 2^n + (A \cdot D + C \cdot B) 2^{n/2} + B \cdot D$

因此，该问题可以分解为：

- 四次 $n/2$ bit 位数的乘法 ($T(n/2)$)
- $A \cdot C$ 左移 n 位 ($\Theta(n)$)
- $A \cdot D + C \cdot B$ 所作的 n 位加法 ($\Theta(n)$)
- $A \cdot D + C \cdot B$ 左移 $n/2$ 位 ($\Theta(n)$)
- 两个 n 位加法合并 ($\Theta(n)$)

因此，该问题可以分解为：

$$\begin{cases} O(1) & n = 1 \\ 4T(n/2) + \Theta(n) & n \geq 2 \end{cases}$$

由主定理可知，算法的时间复杂度仍然为 $\Theta(n^2)$

对算法进行改进，有 $XY = A \cdot C 2^n + (A \cdot D + C \cdot B) 2^{n/2} + B \cdot D = A \cdot C 2^n + ((A+B)(C+D) - AC - BD) 2^{n/2} + B \cdot D$

减少了一个子问题的数量，即该问题分解为：

$$\begin{cases} O(1) & n = 1 \\ 3T(n/2) + \Theta(n) & n \geq 2 \end{cases}$$

由主定理可以得到，算法的时间复杂度为 $\Theta(n^{1.59})$

3.4 排列问题

已知集合 $R = r_1, r_2, \dots, r_n$ ，请设计一个算法生成集合 R 中 n 个元素的全排列，并给出算法时间复杂性分析。

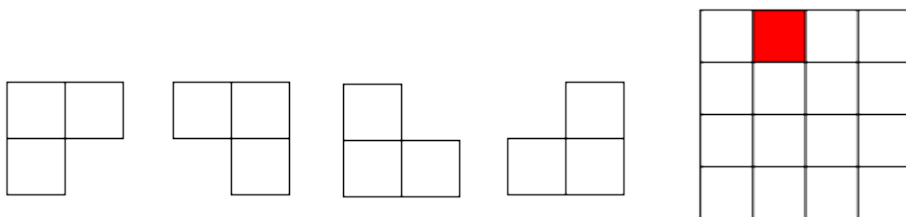
令 $R_j = R - r_j$ ，我们记集合 X 中元素的全排列记为 $perm(X)$ 。那么 $(r_j)perm(X)$ 表示在全排列 $perm(X)$ 的每一种排列前加上前缀 r_j 所得到的排列。

所以， R 的全排列可归纳定义如下：

- 当 $n = 1$ 时， $perm(R) = (r)$ ，其中 r 是集合 R 中唯一的元素；
- 当 $n > 1$ 时， $perm(R)$ 由 $(r_1)perm(R_1), (r_2)perm(R_2), \dots, (r_n)perm(R_n)$ 构成。

3.5 棋盘覆盖问题

在一个 $2^k \times 2^k$ (k 为一正整数) 个方格组成的棋盘中，恰有一个方格与其他方格不同，称该方格为一特殊方格 (红色表示)，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的 4 种不同形态的 L 型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖。



Algorithm 11 Permute

Require: a : 待全排序的数组, k : 起始索引, m : 结束索引

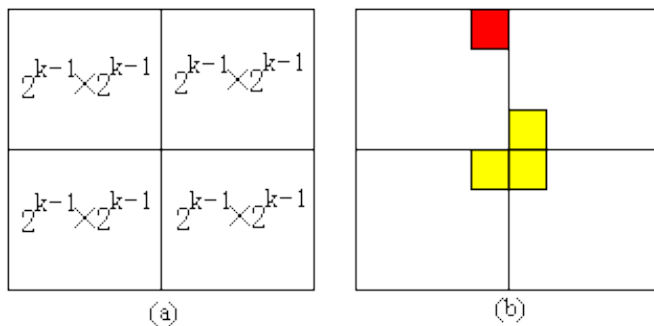
Ensure: 全排列

```

1: if  $k = m$  then
2:   for  $i \leftarrow 0$  to  $\text{length}(a) - 1$  do
3:     print  $a[i]$ 
4:   end for
5:   print new line
6: else
7:   for  $i \leftarrow k$  to  $m$  do
8:     swap  $a[k]$  and  $a[i]$ 
9:     PERMUTE( $a, k + 1, m$ )
10:    swap  $a[k]$  and  $a[i]$ 
11:   end for
12: end if

```

可以将 $2^k \times 2^k$ 棋盘分割为 4 个 $2^{k-1} \times 2^{k-1}$ 子棋盘所示。特殊方格必位于 4 个较小子棋盘之一中，其余 3 个子棋盘中无特殊方格。为了将这 3 个无特殊方格的子棋盘转化为特殊棋盘，可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处，从而将原问题转化为 4 个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘 1×1



题 1 ▶ 给定一个整型数组 $A[1 \dots n]$, x 为一整数。请使用分治策略设计一个算法，该算法能够求解 x 在数组 A 中出现的频度。给出时间复杂度分析。

Algorithm 12 FrequencyX(A, x, l, r)

Require: 需要查询的数组 $A[1..n]$, 需要查找频度的整型数 x , 查询的起始索引 l , 查询的终止索引 r

Ensure: x 在数组 A 的 $[1, n]$ 区间内出现的频度

```

1: if  $l = r$  then
2:   if  $a[l] = x$  then
3:     return 1
4:   else
5:     return 0
6:   end if
7: end if
8: if  $l < r$  then
9:    $mid = \lfloor (l + r) / 2 \rfloor$ 
10:  return FrequencyX( $A, x, l, mid$ ) + FrequencyX( $A, x, mid + 1, r$ )
11: end if

```

调用 FrequencyX($A, x, 1, n$) 即可得到答案。

由算法流程可以得到

$$T(n) = \begin{cases} \Theta(1), & n = 1 \\ 2T(n/2) + \Theta(1), & n > 1 \end{cases}$$

根据主定理, $T(n) = \Theta(n)$

题 2 ▶ 假设 $A[1..n]$ 是一个包含 n 个整数的已排序数组 (非升序)。给定一个整数 x , 现使用分支策略来设计一个算法, 该算法可用来确定 A 中是否存在这样两个整数, 它们的和恰好等于 x , 请你给出该算法, 并分析该算法的时间复杂度。

Algorithm 13 sumX(A, l, r)

Require: 已经排序 (非升序) 的整数数组 $A[1..n]$, 整数 x

Ensure: 若 A 中存在两个整数的和恰好等于 x , 返回 true; 否则返回 false

```

1: if  $l = r$  or  $A[l] + A[r] > x$  then
2:   return false
3: else if  $A[l] + A[r] = x$  then
4:   return true
5: else if  $A[l] + A[r] < x$  then
6:   return SUMX( $A, l + 1, r$ )
7: else
8:   return SUMX( $A, l, r - 1$ )
9: end if

```

调用 sumX($A, 1, n$) 返回即可。

题 3 ▶ 假设 $A[1..n]$ 是一个包含 n 个各异整数的已排序数组 (升序)。给定一个整数 x

4 动态规划

动态规划的实质是**分治**和**消除冗余**，是一种将问题实例分解为更小的、相似的子问题，利用已经求解过的子问题的解来构建更大规模问题的解，避免了重复计算，来解决最优化问题的算法策略。

基本步骤：

1. 找到**最优解**的性质，刻画其结构特征
2. 递归地定义**最优值**
3. 自底向上的方式计算出**最优值**

4.1 矩阵链相乘

问题描述 给定 n 个连乘的矩阵 $M_1 \cdot M_2 \cdot M_{n-1} \cdot M_n$ ，问：所需要的最小乘法次数（最优值）是多少次？对应此最小乘法次数，矩阵是按照什么方式结合相乘（最优解）的？

分析 矩阵 $A_{p \times q} \cdot B_{q \times r}$ 所需要的乘法次数为： $p \times q \times r$

对于多个矩阵连乘的情况，相乘结合的方式不同，所需要的乘法次数也大不相同

对于下面三个矩阵相乘的情况：

$$A_{n \times m} \cdot B_{m \times p} \cdot C_{p \times q}$$

其乘法顺序可以为 $(A \cdot B) \cdot C$ ，其相乘次数为 $n \times m \times p + n \times p \times q$

也可以为 $A \cdot (B \cdot C)$ ，其相乘次数为 $m \times p \times q + n \times m \times q$

动态规划 输入： $r[1 \dots n]$ ，其中矩阵 M_i 为 $(M_i)_{r_i \times r_{i+1}}$

1. 状态定义：

定义 $C[i, j]$ 为计算 $M_i \cdots M_{i+1} \cdots M_{j-1} \cdot M_j$ 所需要的最小乘法次数，则原问题的解为 $C[1, n]$ 。

2. 初始状态：

由定义可知， $C[i, i] = 0$ ，其中 $1 \leq i \leq n$

3. 状态转移方程：

由于 $M_{i,j} = (M_i \cdot M_{i+1} \cdots M_{k-1}) \cdot (M_k \cdot M_{k+1} \cdots M_{j-1})$

而 $M_{i,j} = C[i, j]$, $(M_i \cdot M_{i+1} \cdots M_{k-1}) = C[i, k-1]$, $(M_k \cdot M_{k+1} \cdots M_{j-1}) = C[k, j]$

那么 $C[i, j] = C[i, k-1] + C[k, j] + r_i r_k r_{j+1}$

由于要求的是最小乘法次数，那么

$$C[i, j] = \min_{i < k \leq j} \{C[i, k-1] + C[k, j] + r_i r_k r_{j+1}\}$$

按照更新顺序，以目标为 $C[1, 5]$ 为例，首先需要对 $C[i, i]$ 初始化为 0，其次，要更新 $C[1, 2] = C[1, 1] + C[2, 2] + r_1 r_2 r_3$ ，同理，该层需要更新 $C[2, 3], C[3, 4], C[4, 5]$ 。对于下一层，需要更新 $C[1, 3], C[2, 4], C[3, 5]$ 。以此类推，因此设置循环变量 len 用来表示迭代的层数。

Algorithm 14 矩阵链相乘最小乘法次数

Require: $r[1..n+1]$, 表示 n 个矩阵规模的 $n+1$ 个整数.

Ensure: n 个矩阵连乘的最小乘法次数.

```

1: for  $i \leftarrow 1$  to  $n$  do
2:    $C[i][i] \leftarrow 0$  {填充对角线  $d_0$ }
3: end for
4: for  $d \leftarrow 1$  to  $n-1$  do
5:   for  $i \leftarrow 1$  to  $n-d$  do
6:      $j \leftarrow i+d$  {对角线上  $j, i$  满足的关系}
7:      $C[i, j] \leftarrow \infty$  {将  $C[i, j]$  初始化到无穷大, 防止无效更新}
8:     for  $k \leftarrow i+1$  to  $j$  do
9:        $C[i, j] \leftarrow \min\{C[i, j], C[i, k-1] + C[k, j] + r[i] \times r[k] \times r[j+1]\}$  {状态转移方程}
10:    end for
11:  end for
12: end for
13: return  $C[1, n]$ 

```

算法分析

对状态转移方程那一步的迭代次数进行分析, 容易得到 $T(n) = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=i+1}^j c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=i+1}^{i+d} c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=1}^d c =$

$$c \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} c = c \left(\sum_{i=1}^{n-1} 1 + \sum_{i=1}^{n-2} 2 + \sum_{i=1}^{n-3} 3 + \cdots + \sum_{i=1}^{n-(n-1)} (n-1) \right) = c((n-1) + 2(n-2) + 3(n-3) + \cdots + (n-(n-1))(n-1)) =$$

$$c(n + 2n + 3n + \cdots + n(n-1) - 1^2 - 2^2 - \cdots - (n-1)^2) = c(n(\sum_{i=1}^{n-1} i) - \sum_{i=1}^{n-1} i^2) = c(n \cdot \frac{n(n-1)}{2} - \frac{1}{6}(n-1)n(2n-1)) =$$

$$\frac{1}{6}(cn^3 - cn) = \Theta(n^3)$$

4.2 平面凸多边形最优三角划分

问题描述 给定平面凸多边形 $P = v_0, v_1, \dots, v_n$, 我们希望找到一种最优的三角划分方式, 使得划分后的三角形边长之和最小。

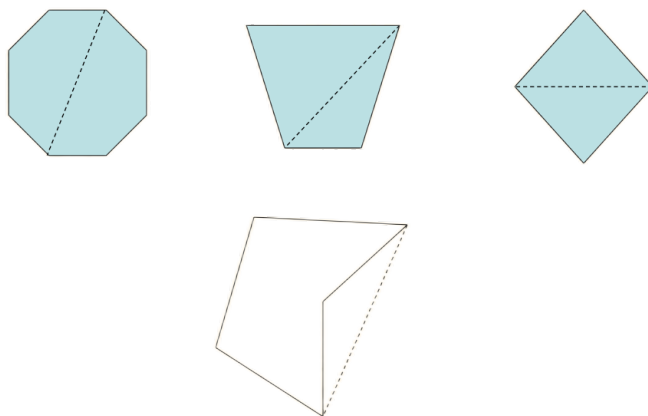
具体要求:

- 划分的三角形的顶点必须是凸多边形的顶点。
- 三角形之间不能有交叉或共享的边或顶点。
- 凸多边形的所有顶点都必须被使用。

我们的目标是找到一种最优的三角划分方式, 使得划分后的三角形边长之和最小。

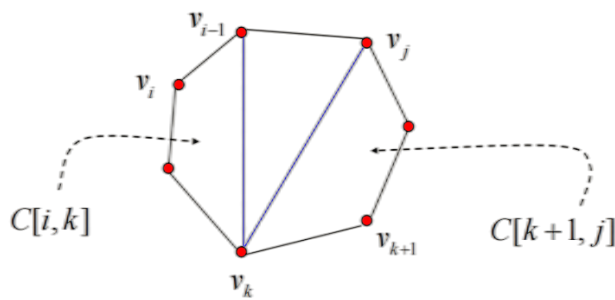
注：

- 平面多边形：由在同一平面上，且不在同一直线上的多条线段首尾顺次连结且不相交所组成的图形称为平面多边形。
- 平面凸多边形：
 - 弦：连接平面多边形的任意两个不同顶点的线段。
 - 平面凸多边形：如果一个平面多边形的任意一条弦，要么在该多边形的内部，要么恰好为该多边形的边，那么，称该平面多边形为凸的。否则，称该平面多边形为凹的。
- 三角划分：将平面凸多边形分割成互不相交的三角形。



- 平面凸多边形的表示：用平面凸多边形顶点的逆时针序列表示凸多边形，即 $P = \{v_0, v_1, \dots, v_n\}$ 表示具有 $n + 1$ 条边的平面凸多边形

动态规划状态定义 定义 $C[i, j]$ 为子凸多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角划分所对应的权函数值，即其最优值。



状态转移方程 假设我们需要计算子凸多边形 $\{v_i, v_{i+1}, \dots, v_j\}$ 的最优三角划分所对应的权函数值，可以考虑将其划分为两个部分： $\{v_i, v_{i+1}, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_j\}$ ，其中 $i \leq k < j$ 。

根据最优子结构性质，我们可以将问题划分为两个子问题，即计算子凸多边形 $\{v_i, v_{i+1}, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_j\}$ 的最优三角划分所对应的权函数值。然后将两个子问题的最优值加上连接两个子多边形的边的长度，即 $w(v_i v_k v_j)$ ，得到整个子凸多边形的最优值。

因此，我们可以得到状态转移方程如下：

$$C[i, j] = \min_{i \leq k < j} \{C[i, k] + C[k, j] + w(v_i v_k v_j)\}$$

其中， $C[i, j]$ 表示子凸多边形 $\{v_i, v_{i+1}, \dots, v_j\}$ 的最优三角划分所对应的权函数值。

添加初始状态则有

$$C[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{C[i, k] + C[k, j] + w(v_i v_k v_j)\} & i < j \end{cases}$$

4.3 0-1 背包问题

问题描述 给定 n 个物品 $\{u_1, u_2, \dots, u_n\}$ 和一个背包，物品 i 的重量为 w_i ，价值为 v_i ，已知背包的承重量为 C 。问：在不撑破背包的条件下，选择哪些物品装入背包，得到的总价值最大？

注：

之所以称为 0-1 背包问题，是因为一个物品要么装入、要么不装入，这两种状态分别用 1 和 0 表示

形式化描述 给定 $C > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$ ，找出一个 n 元的 0-1 向量 $(x_1, x_2, \dots, x_n)$ ， $x_i \in \{0, 1\}$ ， $1 \leq i \leq n$ ，求如下优化问题：

$$\max \sum_{i=1}^n v_i x_i \quad s.t. \sum_{i=1}^n w_i x_i \leq C, x_i \in \{0, 1\}, 1 \leq i \leq n$$

动态规划

1. 状态定义

令 $V[i, j]$ 表示从前 i 个物品中取出一部分放入承重量为 j 的背包所能取得的最大价值，则原问题的解为 $V[n, C]$ 。

2. 初始状态

按照状态定义， $V[0, j] = 0, V[i, 0] = 0$

3. 状态转移方程

对于第 i 个物品，我们有两种选择：放入背包或不放入背包。如果我们选择放入背包，那么当前背包的总价值就是放入这个物品后的总价值；如果我们选择不放入背包，那么当前背包的总价值就是前 $i-1$ 个物品中能够达到的最大总价值。这个状态转移方程的条件是：当物品的重量 $w[i]$ 小于等于当前背包容量 j 时，我们才能考虑放入物品。否则，我们只能选择不放入物品。

$$V[i, j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ V[i-1, j] & j < w_i \\ \max\{V[i-1, j], V[i-1, j-w_i] + v_i\} & i > 0 \text{ and } j \geq w_i \end{cases}$$

算法实现 伪代码如下所示：

Algorithm 15 0-1 背包问题

Require: 物品集合 $\{u_1, u_2, \dots, u_n\}$, 重量分别为 $w_1, w_2, \dots, w_n$, 价值分别为 $v_1, v_2, \dots, v_n$, 承重量为 C 的背包

Ensure: 背包所能装物品的最大价值

```

1: for  $i \leftarrow 0$  to  $n$  do
2:    $V[i, 0] \leftarrow 0$ 
3: end for
4: for  $j \leftarrow 0$  to  $C$  do
5:    $V[0, j] \leftarrow 0$ 
6: end for
7: for  $i \leftarrow 1$  to  $n$  do
8:   for  $j \leftarrow 1$  to  $C$  do
9:      $V[i, j] \leftarrow V[i-1, j]$ 
10:    if  $w_i \leq j$  then
11:       $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-w_i] + v_i\}$ 
12:    end if
13:  end for
14: end for
15: return  $V[n, C]$ 

```

4.4 最长公共子序列问题

问题描述 给定两个定义在字符集 Σ 上的字符串 A 和 B , 长度分别为 n 和 m , 现在要求它们的最长公共子序列的长度值 (最优值), 以及对应的子序列 (最优解)。

概念补充:

$A = a_1 a_2 \dots a_n$ 的一个子序列是形如下式的一个字符串 $a_{i_1} a_{i_2} \dots a_{i_k}$, 其中 $1 \leq i_1 < i_2 < \dots < i_k \leq n$

例如: $A = zxy$, 子序列可以是: “”, z, x, y, zx, zy, xy, zxy 但是 xz, yz, xyz 不是它的子序列。

容易得到, 串的长度为 n 时, 长度为 m 的子序列的个数为 C_n^m , 则总子序列个数为:

$$C_n^0 + C_n^1 + \dots + C_n^n = 2^n$$

暴力枚举 可以尝试利用暴力枚举寻找问题的一般思路。

1. 找出 A 字符串所有可能的子序列 $\Theta(2^n)$;
2. 对于 A 的每一个子序列, 判断其是否是 B 的一个子序列, 需要的时间为 $\Theta(m)$;
3. 求 max; 总的时间为 $\Theta(m2^n)$

动态规划 利用动态规划解决问题。

1. 状态定义

定义 $C[i, j]$ 为 A 序列以 a_i 为结尾的序列与 b_j 为结尾的序列的最长公共子序列的长度值。

2. 初始状态

容易定义, $C[i, 0], C[0, j] = 0$, 也就是说, 空串的匹配结果为长度 0。

3. 状态转移方程

如果 a_i 与 b_j 相同, 那么最长公共子序列可以添加 1 个长度, 即 $C[i, j] = C[i - 1, j - 1] + 1$

如果 a_i 与 b_j 不相同, 要求最长公共子序列, 此时没有增量, 只能利用已有的子问题的解更新答案, 由于要求的是最长公共子序列, 新的 a_i 与 b_j 不能作为增量更新, 那么 $C[i, j] = \max C[i, j - 1], C[i - 1, j]$ 。进而得到:

$$C[i, j] = \begin{cases} 0 & \text{if } i == 0 \text{ or } j == 0 \\ C[i - 1][j - 1] + 1 & \text{if } a_i = b_j \text{ and } i > 0 \text{ and } j > 0 \\ \max(C[i - 1][j], C[i][j - 1]) & \text{if } a_i \neq b_j \text{ and } i > 0 \text{ and } j > 0 \end{cases}$$

4.5 整数划分问题

问题描述 将正整数 K 表示为 p 个整数的和: $K = k_1 + k_2 + \dots + k_p$, 其中 $k_1 \geq k_2 \geq \dots \geq k_p \geq 1$ 。

正整数 K 的这种表示, 叫做正整数划分。正整数 K 的不同划分的个数叫做划分数, 记作 $p(K)$ 。

请设计一个算法, 对于一个给定的正整数 (输入), 求出该整数的划分数 (输出)

动态规划

1. 状态定义

在正整数 K 的所有划分, 我们将最大加数 k_1 不大于 m 的划分数记作 $p(K, m)$, 则问题的答案为 $p(K, K)$

譬如对于 6, 有如下划分:

6; ——最大加数为 6

5 + 1; ——最大加数为 5

4 + 2; 4 + 1 + 1; ——最大加数为 4

3 + 3; 3 + 2 + 1; 3 + 1 + 1; ——最大加数为 3

2 + 2 + 2; 2 + 2 + 1 + 1; 2 + 1 + 1 + 1; ——最大加数为 2

1 + 1 + 1 + 1 + 1 + 1; ——最大加数为 1

由定义可以知道 $p(6, 1) = 1, p(6, 2) = 4, p(6, 3) = 7, p(6, 4) = 9, p(6, 5) = 10, p(6, 6) = 11$

2. 初始状态

显然 $p(i, 0) = 0, p(i, 1) = 1$

3. 对于 $p(i, j)$, 即将正整数 i 划分为最大加数不大于 j 的正整数和的划分数, 可以通过以下规则进行转移:

当 $m > K$ 时, 即表示要将 K 划分为最大加数不大于 m 的正整数和, 而 $m > K$, 事实上这种情况不存在有效的划分方式, 因此 $p(K, m)$ 等于 $p(K, K)$, 即最大加数不大于 K 的划分数。

当 $m = K$ 时, 表示要将 K 划分为最大加数不大于 K 的正整数和, 只有一种划分方式, 即划分为一个 K 。因此, $p(K, m) = p(K, K - 1) + 1$ 。

当 $1 < m < K$ 时, 表示要将 K 划分为最大加数不大于 m 的正整数和, 有两种情况:

- 第一种情况是将 K 划分为最大加数为 m 的正整数和，分为两块，一块为正整数 m ，另一块为 $K - m$ 的组合，即将 $K - m$ 划分为最大加数不大于 m 的正整数和，并且将所有划分中的每个数都加上 m 。
- 第二种情况是将 K 划分为最大加数不大于 $m - 1$ 的正整数和，即将 K 划分为最大加数不大于 $m - 1$ 的正整数和的划分数。

因此， $p(K, m) = p(K - m, m) + p(K, m - 1)$

则

$$p(K, m) = \begin{cases} 1 & m = 1 \\ p(K, K) & m > K \\ p(K, K - 1) + 1 & , m = K \\ p(K - m, m) + p(K, m - 1) & , 1 < m < K \end{cases}$$

4.6 集合划分

问题描述 给定一个集合 X ，且集合中元素互不相同。集合 P 由 X 的子集构成。若 P 中的元素满足以下性质：

1. P 中的任一元素不是空集。
2. P 中的元素的并集等于 X 。
3. P 中的任何两个元素的交集为空。

那么称 P 是 X 的一个划分。

以集合 $X = \{1, 2, 3, 4\}$ 为例：

$P = \{\{1, 2, 3, 4\}, \{\}\}$ 不是 X 的划分，因为存在空集

$P = \{\{1, 2\}, \{3\}\}$ 不是 X 的划分，因为子集的并不是原集合

$P = \{\{1, 2, 3\}, \{2, 4\}\}$ 也不是 X 的划分，因为两子集的交集不是空集

$P = \{\{1, 2\}, \{3, 4\}\}$ 符合上述三点，是 X 的一个划分

设计一个算法，求一个包含 n 个不同元素的集合，一共有多少种划分？

动态规划

1. 状态定义

用 $F(i, j)$ 表示一个包含前 i 个元素的集合，划分为 j 个子集所具有的划分数。那么原问题的解为 $F(n, 1) + F(n, 2) + \dots + F(n, n)$

2. 初始状态

$F(i, 1) = 1, F(i, 0) = 0, F(0, i) = 0$

3. 状态转移方程

$F(i, j)$ 表示一个包含前 i 个元素的集合，划分为 j 个子集所具有的划分数。讨论以下三种情况：

- (a) 当 $i < j$ 时，划分数大于集合的元素数量，按照定义，不能存在空集，因此，这样的划分为 0
- (b) 当 $i = j$ 时，划分数刚好等于集合的元素数量，只需将每个元素放入一个单独的集合中即可，这样的划分有且仅有一个，即 $F(i, i) = 1$

(c) 当 $i > j$ 时，划分数目小于集合的元素数量。

考虑第 i 个元素的放置情况，若第 i 个元素单独放入一个空集当中，则该集合为第 j 个集合，因此该划分数目刚好等于 $F(i-1, j-1)$

若第 i 个元素随意放入 j 个集合中，一共有 j 种选择，则该划分数目为 $j \cdot F(i-1, j)$

则 $F(i, j) = F(i-1, j-1) + j \cdot F(i-1, j)$

则

$$F(i, j) = \begin{cases} 1 & i = 1 \text{ or } i = j \\ 0 & i < j \text{ or } i = 0 \text{ or } j = 0 \\ F(i-1, j-1) + j \cdot F(i-1, j) & i > j \text{ and } i > 1, j > 1 \text{ and } i \neq j \end{cases}$$

4.7 编辑距离

问题描述 设 A 和 B 是 2 个字符串。要用最少的字符操作将字符串 A 转换为字符串 B 。字符串操作包括：

1. 删除一个字符；
2. 插入一个字符；
3. 将一个字符改为另一个字符。

将字符串 A 变换为字符串 B 所用的最少操作次数称为字符串 A 到 B 的编辑距离，记为 $d(A, B)$ 。请设计一个动态规划算法，对于给定的任意两个字符串 A 和 B ，计算其编辑距离 $d(A, B)$ 。

动态规划

1. 状态定义：

假设 $A = a_1a_2 \dots a_n$ 的长度为 n ， $B = b_1b_2 \dots b_m$ 的长度为 m 。令 $d[i, j]$ 表示 $a_1a_2 \dots a_i$ 和 $b_1b_2 \dots b_j$ 之间的编辑距离。

2. 初始状态：

在 $d[i, j]$ 中，对于 $i = 0$ 或 $j = 0$ 的情况，只要一直执行插入操作即可。即 $d[i, 0] = i, d[0, j] = j$

3. 状态转移方程：

对于 $d[i, j]$ ，讨论 A 的第 i 个字符和 B 的第 j 个字符的情况

(a) 若 A 的第 i 个字符与 B 的第 j 个字符相同，那么 $d[i, j] = d[i-1, j-1]$

(b) 若 A 的第 i 个字符与 B 的第 j 个字符不同，有以下三种操作：

- 修改操作：将 A 的第 i 个字符修改为 B 的第 j 个字符 $d[i, j] = d[i-1, j-1] + 1$
 - 插入操作：将 A 与 B 的前 $j-1$ 个字符编辑相同，最后再插入字符 b_j 即可，也即 $d[i, j] = d[i, j-1] + 1$
 - 删除操作：将 A 的前 $i-1$ 个字符与 B 编辑相同，最后再删除 a_i 即可，也即 $d[i, j] = d[i-1, j] + 1$
- 由于是取最小值，那么 $d[i, j] = \min\{d[i-1, j-1] + 1, d[i, j-1] + 1, d[i-1, j] + 1\}$

则

$$d[i, j] = \begin{cases} i & j = 0 \\ j & i = 0 \\ d[i-1, j-1] & i > 0, j > 0, a_i = b_j \\ \min\{d[i-1, j-1] + 1, d[i, j-1] + 1, d[i-1, j] + 1\} & i > 0, j > 0, a_i \neq b_j \end{cases}$$

5 贪心策略

贪心策略总是做出在当前看来最好的选择，并且不从整体上考虑最优，所做出的每一步选择只是局部意义上最优选择（因而效率往往较高）。在求解过程中，逐步扩大解的规模，直到问题解决。贪心策略有时能得到最优解，但不能保证一定得到最优解。

虽然贪心策略不能保证对所有问题都能得到最优解，但是对很多问题（包括很多著名的问题）都能产生整体最优解。

注：假设有 4 种硬币，面值分别为 2 角 5 分、1 角、5 分和 1 分。现在要求以最少的硬币个数找给顾客 6 角 3 分。

基于贪心策略：面值越大，需要的个数越少。通常的做法是：先拿两个 2 角 5 分 + 1 个 1 角 + 3 个 1 分。现在假设面值有 1 角 1 分、5 分和 1 分，要找给顾客 1 角 5 分。如果还是使用上述贪心策略，得到的解是：1 个 1 角 1 分 + 4 个 1 分。而实际上最优解是 3 个 5 分。

使用贪心策略的难点在于：证明所设计的算法确实能够正确解决所求解的问题。（正确性证明）

5.1 单源最短路径问题

5.1.1 Dijkstra 算法

• $G = (V, E)$ 是一个有向图，每条边上有一个非负整数表示长度值，其中有一个顶点称为源顶点。所谓的单源最短路径问题就是：求解该源顶点到所有其它顶点的最短路径值。

5.2 最小生成树

设 $G = (V, E)$ 是连通无向带权图， (V, T) 是 G 的一个子图，并且 T 是一颗树，那么称 (V, T) 是 G 的生成树。如果 T 的权之和是所有生成树中最小的，那么则称之为最小生成树。

网络的最小生成树在实际中有广泛应用。例如，在设计通信网络时，用图的顶点表示城市，用边 (v, w) 的权 $c[v][w]$ 表示建立城市 v 城市 w 之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。

首先我们假定 G 是连通的，如果 G 是非连通的，那么，可以对 G 的每个连通子图应用求解最小生成树的算法。

5.2.1 Kruscal 算法

1. 算法思路：

- 对 G 的边 E 按权重以非降序排列。
- 初始时输出树 $T = \{\}$ ；依次取排序表中的每条边（应用贪心策略，权重小的先取），若加入 T 不形成回路，则加入 T ；否则将其丢弃；

- 不断重复步骤 2，直到树 T 包含 $n - 1$ 条边，算法结束。

2. 伪代码：

Algorithm 16 Kruscal 算法

Require: 图的边集合 E 和顶点集合 V

Ensure: 生成树 T

```

1: 对  $E$  中的边按权重以非降序排列  $\{O(m \log m)\}$ 
2: for 每个顶点  $v \in V$  do
3:   MAKESET( $v$ )  $\{\Theta(n)\}$ 
4: end for
5:  $T \leftarrow \{\}$ 
6: while  $|T| < n - 1$  do
7:   令  $(x, y)$  为  $E$  中的下一条边
8:   if FIND( $x$ )  $\neq$  FIND( $y$ ) then
9:     将  $(x, y)$  加入  $T$   $\{\Theta(n)\}$ 
10:    UNION( $x, y$ )  $\{\Theta(n)\}$ 
11:   end if
12: end while

```

该算法通过不断选择边并判断其顶点是否在同一个连通分量中，来构建最小生成树。算法开始时将每个顶点看作是一个独立的连通分量，然后按照边的权重从小到大进行选择，如果边的两个顶点不在同一个连通分量中，就将它们合并到同一个连通分量中，并将边加入生成树 T 中。最终，当生成树 T 中的边数达到 $n - 1$ 时，算法结束，生成的 T 就是原图的最小生成树。

由频度分析法可知，该算法的时间复杂度为 $O(m \log m + n)$

5.2.2 Prim 算法

- 算法思路：设 $G = (V, E)$ 是连通无向带权图， $V = \{1, 2, \dots, n\}$ 。构造 G 的最小生成树的 Prim 算法的基本思想是：
 - 首先将一个顶点 1 添加到集合 X 中作为初始点，表示最小生成树中的第一个顶点。
 - 当集合 X 不包含图中的所有顶点时，执行以下操作：
 - 在所有的边中选择权重最小的边 (x, y) ，其中 $x \in X$ 且 $y \in Y$ ， X 表示已经选择的顶点集合， Y 表示未选择的顶点集合。
 - 将边 (x, y) 加入当前的最小生成树。
 - 将顶点 y 从集合 Y 移动到集合 X 中，表示将顶点 y 添加到最小生成树中。
 - 重复步骤 2，直到集合 X 包含了图中的所有顶点，即 $X = V$ 。

在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树

2. 伪代码

Algorithm 17 Prim 算法

Require: 图 $G = (V, E)$, 顶点集合 $V = \{1, 2, \dots, n\}$, 边的权重 $cost[1 \dots n]$

Ensure: 图的最小生成树 T

```

1:  $T \leftarrow \{\}$  {存储最小生成树的边集合}
2:  $X \leftarrow \{1\}$  {已选择的顶点集合, 初始包含顶点 1}
3:  $Y \leftarrow V - \{1\}$  {未选择的顶点集合, 初始为除顶点 1 外的所有顶点,  $\Theta(n)$ }
4: 初始化数组  $N$  和  $C$  { $C$  数组用于记录顶点边的权重,  $N$  数组用于记录最小生成树中与顶点相连的边的起点}
5: for  $y \leftarrow 2$  to  $n$  do
6:   if  $y$  邻接于 1 then
7:      $N[y] \leftarrow 1$  { $O(n)$ }
8:      $C[y] \leftarrow cost[1, y]$ 
9:   else
10:     $C[y] \leftarrow \infty$ 
11:   end if
12: end for
13: for  $j \leftarrow 1$  to  $n - 1$  do
14:   找到  $y \in Y$  使得  $C[y]$  最小 { $\Theta(n^2)$ }
15:    $T \leftarrow T \cup \{(y, N[y])\}$ 
16:    $X \leftarrow X \cup \{y\}$ 
17:    $Y \leftarrow Y - \{y\}$ 
18:   for 每个邻接于  $y$  的顶点  $w \in Y$  do
19:     if  $cost[y, w] < C[w]$  then
20:        $N[w] \leftarrow y$  { $\Theta(mn)$ }
21:        $C[w] \leftarrow cost[y, w]$ 
22:     end if
23:   end for
24: end for
25: return  $T$ 

```

算法的时间复杂度为 $\Theta(n^2 + nm)$

5.3 哈夫曼编码

哈夫曼编码是一种用于数据压缩的有效编码方法, 它可以根据字符在文件中出现的频率表, 构建一个用 0 和 1 串表示字符的最优编码方式。该算法采用贪心策略, 将出现频率高的字符用较短的编码表示, 而出现频率低的字符用较长的编码表示, 以实现总码长的缩短。

在哈夫曼编码中, 采用前缀码的方式进行编码。前缀码的特点是任一字符的编码不是其他字符编码的前缀, 这样在译码时可以通过从编码串的开头开始逐步匹配, 快速准确地还原出原始字符。

通过构建哈夫曼树来生成哈夫曼编码。首先, 根据字符出现的频率建立一个字符频率表。然后, 根据频率表构建一个哈夫曼树, 其中频率较低的字符作为叶子节点, 频率较高的字符作为内部节点。在构建哈夫曼树的过程中, 使用贪心算法选择频率最低的两个节点合并, 直到只剩下一个根节点。最后, 通过从根节点开始遍历哈夫曼树, 向左

走为 0，向右走为 1，得到每个字符的哈夫曼编码。

由于哈夫曼编码具有前缀性质，译码过程非常简单。只需从编码串的开头开始逐个字符匹配，根据匹配结果不断向下遍历哈夫曼树，直到找到对应的字符，即可还原原始数据。

哈夫曼编码在数据压缩中具有广泛应用，能够显著减小数据的存储空间和传输带宽，提高数据传输和存储效率。

Algorithm 18 Huffman 编码

Require: n 个字符的集合 $C = \{c_1, \dots, c_n\}$ 及其频度 $\{f(c_1), \dots, f(c_n)\}$

Ensure: C 的 Huffman 树 (V, T)

根据频度 (作为键值) 将所有字符插入最小堆 H

$V \leftarrow C; T \leftarrow \{\}$

for $j \leftarrow 1$ to $n - 1$ **do**

$c \leftarrow \text{DeleteMin}(H)$

$c' \leftarrow \text{DeleteMin}(H)$

$f(v) \leftarrow f(c) + f(c')$

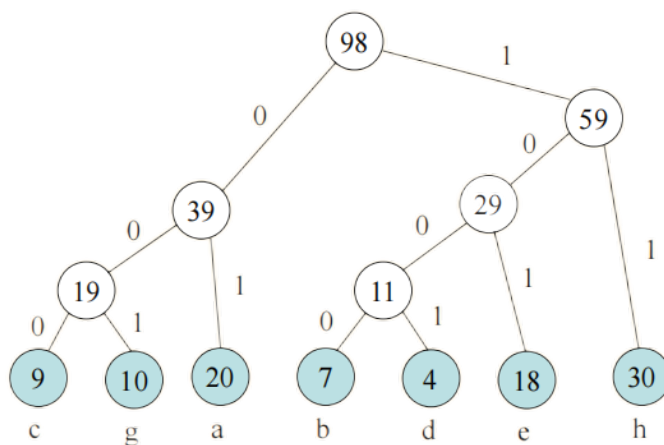
$\text{INSERT}(H, v)$

$V \leftarrow V \cup \{v\}$

$T \leftarrow T \cup \{(v, c), (v, c')\}$

end for

题 1 ▶ 假设一个文件由字符 a, b, c, d, e, g, h 组成。每个字符的出现次数为 $f(a) = 20, f(b) = 7, f(c) = 10, f(d) = 4, f(e) = 18, f(g) = 10, f(h) = 30$ 试画出相应的哈夫曼编码树，并给出每个字符的哈夫曼编码。



具体构造过程见数据结构

6 图遍历

6.1 深度优先遍历

深度优先搜索 (Depth-First Search, DFS) 是一种常用的图遍历算法，用于遍历或搜索图或树的所有节点。该算法以深度为优先，即尽可能深地探索每个分支，直到到达叶子节点或无法继续向下探索的节点，然后回溯到前一层继续探索其他分支。

下面介绍深度优先搜索的一些基本概念：

深度优先搜索生成树 (Depth-First Search Spanning Tree) 深度优先搜索生成树是通过深度优先搜索算法构建的一棵树。给定一个图，深度优先搜索从一个起始顶点开始，沿着路径尽可能深入地访问未被访问的顶点，直到无法再深入为止。生成树是由搜索过程中访问到的顶点和边所构成的子图，其中每个顶点都可以通过一条路径与起始顶点相连。生成树中不包含回路。

深度优先搜索生成森林 (Depth-First Search Spanning Forest) 深度优先搜索生成森林是在一个连通图中进行深度优先搜索时得到的多个深度优先搜索生成树的集合。如果图是非连通的，那么生成森林由多个独立的生成树组成，每个生成树对应图中的一个连通分量。

predfn (Pre Depth-First Number) predfn 是在深度优先搜索生成树 (森林) 中顶点的先序号。先序号是指按照深度优先搜索的先序方式访问该生成树时，给每个顶点分配的顺序号。具体而言，它表示顶点在生成树中被首次访问的顺序。

postdfn (Post Depth-First Number) postdfn 是在深度优先搜索生成树 (森林) 中顶点的后序号。后序号是指按照深度优先搜索的后序方式访问该生成树时，给每个顶点分配的顺序号。具体而言，它表示顶点在生成树中被最后一次访问的顺序。

对边 (v, w) 进行探测 在深度优先搜索过程中，当探索到一条边 (v, w) 时，进行探测的目的是检查顶点 w 是否已经被访问过。这是为了避免访问已经访问过的顶点，防止进入循环或重复访问同一个顶点。通过探测边的终点顶点，可以确定是否需要访问该顶点或继续深度优先搜索的过程。

下面是深度优先搜索算法的一般步骤：

- 选择一个起始节点作为当前节点，并将其标记为已访问。
- 对于当前节点，依次访问它的邻居节点。
- 如果邻居节点未被访问过，将其标记为已访问，并将其设置为当前节点，然后重复步骤 2。
- 如果所有邻居节点都已被访问过，回溯到上一层节点，继续访问其他未被访问的邻居节点。
- 重复步骤 3 和步骤 4，直到所有节点都被访问过或无法继续访问。

深度优先搜索可以使用递归或栈（堆栈）来实现。在递归实现中，算法会自动使用系统栈来存储每个节点的信息。在使用栈的非递归实现中，我们可以手动维护一个栈数据结构来存储待访问的节点。

深度优先搜索算法的应用广泛，包括解决迷宫问题、拓扑排序、生成最小生成树、图的连通性检测等。它也是其他高级算法的基础，如深度优先遍历和回溯算法。

Algorithm 19 深度优先遍历

Require: 无向图或有向图 $G = (V, E)$

Ensure: 深度优先搜索树 (森林) 中每个顶点的先序号、后序号

```

1:  $predfn \leftarrow 0, postdfn \leftarrow 0$ 
2: for each  $v \in V$  do
3:    $visited[v] \leftarrow false$ 
4: end for
5: for each  $v \in V$  do
6:   if  $visited[v] = false$  then
7:      $DFS(v)$ 
8:   end if
9: end for
10:
11:  $DFS(v)$ :
12:  $visited[v] \leftarrow true$ 
13:  $predfn \leftarrow predfn + 1$ 
14: for each  $(v, w) \in E$  do
15:   if  $visited[w] = false$  then
16:      $DFS(w)$ 
17:   end if
18: end for
```

- 树边 (Tree edges) - 深度优先搜索生成树中的边：探测边 (v, w) 时， w 是 “unvisited” 状态，则边 (v, w) 是树边。
- 回边 (Back edges) - 在迄今为止所构建的深度优先搜索生成树中， w 是 v 的祖先，并且在探测 (v, w) 时， w 已经被标记为 “visited”，则 (v, w) 为回边。
- 前向边 (Forward edges) - 在迄今为止所构建的深度优先搜索生成树中， w 是 v 的后裔，并且在探测 (v, w) 时， w 已经被标记为 “visited”，则 (v, w) 为前向边。
- 横跨边 (Cross edges) - 所有其他的边。

6.2 广度优先遍历

广度优先搜索的思路是在访问一个顶点 v 后，接下来依次访问邻接于 v 的所有顶点。对应的搜索树称之为广度优先搜索生成树。

具体实现可以使用**队列** (Queue)

- 在无向图中，边分为：树边或者是横跨边。
- 在有向图中，边分为：树边，回边及横跨边。不存在前向边

Algorithm 20 广度优先搜索

Require: 无向图或有向图 $G = (V, E)$

Ensure: 广度优先搜索树中每个顶点的编号

```

1:  $bfn \leftarrow 0$ 
2: for  $v \in V$  do
3:    $visited[v] \leftarrow \text{false}$ 
4: end for
5: for  $v \in V$  do
6:   if  $visited[v] = \text{false}$  then
7:      $BFS(v)$ 
8:   end if
9: end for
10:
11:  $bfs(v)$ 
12:  $Q \leftarrow \{v\}$ 
13:  $visited[v] \leftarrow \text{true}$ 
14:  $bfn \leftarrow 0$ 
15: while  $Q \neq \{\}$  do
16:    $v \leftarrow \text{Pop}(Q)$ 
17:    $bfn \leftarrow bfn + 1$ 
18:   for each  $(v, w) \in E$  do
19:     if  $visited[w] = \text{false}$  then
20:        $\text{Push}(w, Q)$ 
21:        $visited[w] \leftarrow \text{true}$ 
22:     end if
23:   end for
24: end while

```

7 回溯法和分支限界法

7.1 回溯法

回溯法 (Backtracking) 是一种常用于解决组合优化问题的算法。它通过穷举**搜索**的方式，在问题的解空间中寻找满足特定条件的解，同时**能避免不必要的搜索**。

回溯法通过不断地进行选择、验证和回退操作，逐步构建解决方案，并在发现不可行解时进行回溯，以找到所有可能的解或找到一个满足条件的解。

回溯法通常适用于以下类型的问题：

- 组合问题：如子集、组合、排列等问题。
- 搜索问题：如图的遍历、迷宫问题等。
- 集合划分问题：将集合划分成若干个子集，使得每个子集的元素满足特定条件。

回溯法按**深度优先策略**搜索问题的解空间树。

算法搜索至解空间树的任意一节点时，先判断该节点是否可能包含问题的解：

1. 如果肯定不包含，则跳过这个节点；
2. 如果可能包含，进入该子树，继续按深度优先策略搜索；
3. 若某节点 i 的所有子节点都不可能包含问题的解，则回溯到 i 的父节点，生成下一个节点，继续搜索。

回溯法的基本框架可以描述为以下步骤：

1. 定义问题的解空间：我们引入以下概念，介绍问题的解空间。

问题的解向量 (Solution Vector) 问题的解向量是指问题的一个解所对应的 n 维向量 $(x_1, x_2, \dots, x_n)$ ，表示了问题的一个可能解。解向量的每个元素 x_i 代表问题的一个自变量或决策变量，其取值范围受到约束条件的限制。

显式约束 (Explicit Constraints) 显式约束是指问题中明确给出的、直接对解向量的分量 x_i 施加的限制条件。这些约束以一种明确的形式表示，并直接约束了解向量中变量的取值范围。

例如，变量的取值必须满足某个范围、满足某个等式或不等式关系等。

隐式约束 (Implicit Constraints) 隐式约束是指问题中间接约束解向量的不同分量的条件，这些约束条件可能不以明确的形式给出，而是通过问题的定义和限制间接地约束了解向量的取值范围。

隐式约束可以通过问题的目标函数、问题的结构、问题的逻辑关系等来体现。

问题的解空间 (Solution Space) 对于问题的一个实例，该问题的解空间是指问题的所有满足**显式约束**条件的所有 n 维向量，构成了该问题实例的一个解空间。解空间包含了满足所有约束条件的解向量，即所有可能的解。解空间的大小取决于问题的定义和约束条件。

在优化问题中，解空间可能是连续的，而在组合问题中，解空间可能是离散的。寻找问题的解就是在解空间中搜索满足约束条件和目标函数要求的 n 维向量。

2. 定义约束条件：确定对解的限制条件，即约束条件。约束条件可以是问题中的限制、条件或规则，用于筛选掉不符合要求的解。

3. 定义目标函数：确定问题的目标函数，即用于评价解好坏的函数。目标函数可以是问题的优化目标或满足特定条件的判断。
4. 实现回溯算法：按照以下步骤实现回溯算法：
 - (a) 选择：从解空间中选择一个候选解的元素，将其添加到当前的解中。
 - (b) 约束条件检测：检测当前解是否满足约束条件。如果不满足，剪枝（即舍弃该解）。
 - (c) 目标函数检测：检测当前解是否满足目标函数的要求。如果满足，输出解或记录解（根据具体问题的要求）。
 - (d) 递归：在满足约束条件的情况下，继续递归调用回溯算法，处理下一个位置或下一层的选择。
 - (e) 撤销选择：在递归返回时，撤销上一步的选择，返回上一层，尝试其他的选择。

回溯法的优点是可以找到所有可能的解，但其缺点是在问题空间较大时，搜索复杂度较高。因此，在实际应用中，需要根据问题的特点进行剪枝操作，减少无效的搜索，提高效率。

总结起来，回溯法是一种通过穷举搜索的方式，在解空间中寻找满足特定条件的解的算法。它的基本思想是选择、验证、回退，通过不断地尝试和回溯，找到所有可能的解或满足特定条件的解。回溯法在组合问题、搜索问题和集合划分问题等方面具有广泛的应用。

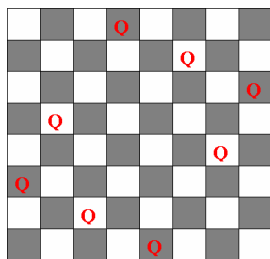
7.1.1 图的 3 着色

给定一个无向图 $G = (V, E)$ ，及 3 种颜色 $\{1, 2, 3\}$ ，现要为图的每个顶点着色。每顶点只能着一种颜色，并且要求相邻的顶点具有不同的颜色。

一个具有 n 个顶点的图，可用一个 n 维的向量 $(c_1, c_2, \dots, c_n)$ 表示一种着色方案， $c_i \in \{1, 2, 3\}$ ， $i = 1, 2, \dots, n$ ，共有 3^n 种可能的着色，可用一棵完全的 3 叉树表示。

7.1.2 n 皇后问题

n 皇后问题是一个经典的组合优化问题，旨在找到在一个 $n \times n$ 的棋盘上放置 n 个皇后，使得它们互相之间不会相互攻击。在国际象棋中，皇后可以在同一行、同一列和对角线上攻击其他棋子。



具体来说， n 皇后问题要求找到一种放置方式，使得每个皇后都位于不同的行和不同的列，且没有两个皇后会互相攻击。换句话说，任意两个皇后不能位于同一行、同一列或同一对角线上。

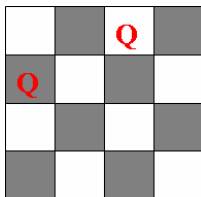
考察 n 皇后问题 $n = 4$ 的情形：解空间有 4^4 (可减少至 $4!$) 种布局，可用一棵高度为 4 的完全 4 叉树表示：树的根对应于没有放置皇后的布局，第一层节点对应于皇后在第一行 (列) 可能放置列 (行) 的情况，依此类推。

我们定义如下布局方式：

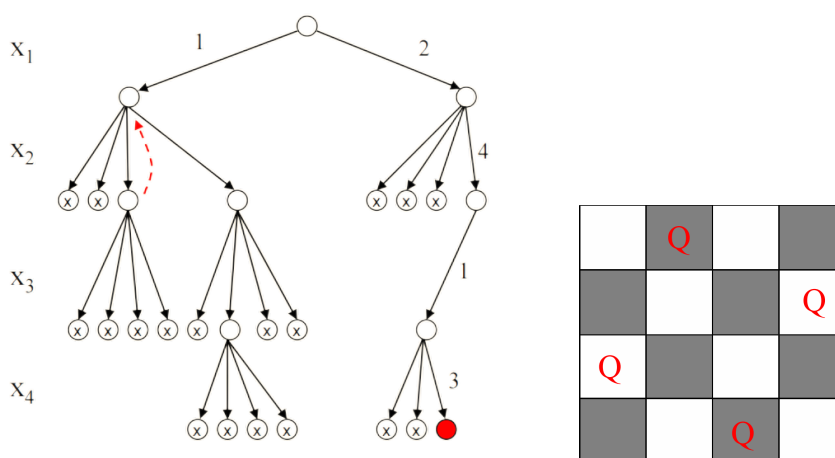
- 合法布局：一个不互相攻击的 4 皇后布局

- 部分布局：一个不互相攻击的少于 4 个皇后的布局

下图表示一个部分布局，用向量 $(3,1,0,0)$ 表示：



通过如下解空间树，可以构造一个合法布局：



讨论：蛮力方法候选解的个数是 $n!$ ；而使用回溯法，候选解的个数是 n^n ，回溯的效率是否太低了？

然而，仔细分析可以发现，回溯法可以极大减少测试次数：

例如，假设前两个皇后均放在了第一列，蛮力方法仍旧要测试 $(n-2)!$ 个候选解；而回溯法只要进行一次测试就可以避免剩余无意义的测试。

所以，尽管在最坏情况下要用 $O(n^n)$ 时间来求解，然而大量实际经验表明，它在有效性上远远超过蛮力方法 $O(n!)$ 。例如在 4 皇后问题中，只搜索了 256 个节点中的 19 个就找到了解。

对于 4 皇后问题，采用递归回溯，得到的算法如下所示：

7.1.3 总结

在回溯法中，每个 x_i 均是属于某个有限集合的，不妨称之为 X_i ，那么回溯法实质上是按照词典顺序考虑笛卡儿积 $X_1 \times X_2 \times \dots \times X_n$ 中的元素。

1. 算法最初从空向量开始，先选择 X_1 中的最小元素作为 x_1 ；
2. 如果 (x_1) 是部分解，那么选择 X_2 中的最小元素作为 x_2 ；
3. 如果 (x_1, x_2) 是部分解，则继续考虑 X_3 中的最小元素作为 x_3 ；否则，考虑 X_2 中的第二个元素作为 x_2 。依此类推。

当算法已经检测到部分解 $(x_1, x_2, \dots, x_j)$ ，需要继续考虑 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 时，有以下几种情形：

Algorithm 21 递归回溯求解 4 皇后问题

Require: 空

Ensure: 4 皇后问题的解向量 $c[1 \dots 4]$

```

1: for  $k \leftarrow 1$  to 4 do
2:    $c[k] \leftarrow 0$ 
3: end for
4: flag  $\leftarrow$  false
5: SetQueen(1)
6: if flag then
7:   输出  $c$ 
8: else
9:   输出”无解”
10: end if
11:
12: SetQueen( $k$ ):
13: for  $x \leftarrow 1$  to 4 do
14:    $c[k] \leftarrow x$ 
15:   if  $c$  为合法布局 then
16:     flag  $\leftarrow$  true 并退出循环
17:   else if  $c$  是部分布局 then
18:     SetQueen( $k + 1$ )
19:   end if
20: end for

```

Algorithm 22 迭代回溯求解 4 皇后问题

Require: 空

Ensure: 4 皇后问题的解向量 $c[1 \dots 4]$

```

1: for  $k \leftarrow 1$  to 4 do
2:    $c[k] \leftarrow 0$ 
3: end for
4:  $\text{flag} \leftarrow \text{false}$ 
5:  $k \leftarrow 1$  {从第一个皇后开始}
6: while  $k \geq 1$  do
7:   while  $c[k] \leq 3$  do
8:      $c[k] \leftarrow c[k] + 1$  {安排第  $k$  个皇后的位置}
9:     if  $c$  为合法布局 then
10:       $\text{flag} \leftarrow \text{true}$  并退出循环
11:     else if  $c$  为部分布局 then
12:       $k \leftarrow k + 1$  {准备安排下一个皇后的位置}
13:     end if
14:   end while {若布局既不是合法布局，也不是部分布局，则试其他位置}
15:    $c[k] \leftarrow 0$  { $c[k]$  试验了所有位置均失败， $c[k]$  只好归 0，回溯}
16:    $k \leftarrow k - 1$  {回溯到上一个顶点 (配合第 6 句的  $k \geq 1$  来理解)}
17: end while
18:
19: 输出  $c$ 

```

- 若 v 表示问题的最终合法解，算法记录下它作为一个解。如果只需要一个解，算法结束；否则，继续找其他解。
- 如果 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 是一个部分解，那么选择集合 X_{j+2} 中未使用过的最小元素作为 x_{j+2} 。（向前搜索）

如果 v 既非最终解，也非部分解，那么会有以下两种情况：

- 如果集合 X_{j+1} 中还有其他未曾使用过的元素，则选择下一个未曾使用过的元素作为 x_{j+1} 。
- 如果集合 X_{j+1} 中没有其他未曾使用过的元素，则回溯，将 X_j 中未曾使用的下一个元素作为 x_j ，并将 x_{j+1} 进行重置。如果集合 X_j 中没有未曾使用的元素，那么继续回溯（同样要进行重置）。

回溯法的基本步骤：

1. 针对所给问题，定义问题的**解空间**；
2. 确定易于搜索的解空间结构；
3. 以**深度优先方式**搜索解空间，并在搜索过程中用**剪枝函数**（如隐约束）避免无效搜索。

Algorithm 23 递归回溯法

Require: 集合 $X_1, X_2, \dots, X_n$ （明确或隐含地定义）

Ensure: 解向量 $v = (x_1, x_2, \dots, x_i, x_n)$, $0 \leq i \leq n$

```

1:  $v \leftarrow ()$ 
2:  $\text{flag} \leftarrow \text{false}$ 
3:  $\text{advance}(1)$ 
4: if  $\text{flag}$  then
5:   输出  $v$ 
6: else
7:   输出”无解”
8: end if
9:
10:  $\text{advance}(k)$ :
11: for  $x \in X_k$  do
12:    $x_k \leftarrow x$  {将  $x_k$  赋值为  $x$ }
13:   将  $x_k$  加入  $v$ 
14:   if  $c$  为最终解 then
15:     设置  $\text{flag} \leftarrow \text{true}$  并退出循环
16:   else if  $v$  为部分解 then
17:      $\text{advance}(k+1)$ 
18:   end if
19: end for
```

Algorithm 24 迭代回溯法

Require: 集合 $X_1, X_2, \dots, X_n$ (明确或隐含地定义)

Ensure: 解向量 $v = (x_1, x_2, \dots, x_i, x_n)$, $0 \leq i \leq n$

```

1:  $v \leftarrow ()$ 
2:  $\text{flag} \leftarrow \text{false}$ 
3:  $k \leftarrow 1$ 
4: while  $k \geq 1$  do
5:   while  $X_k$  没有被穷举完毕 do
6:      $x_k \leftarrow X_k$  中的下一个元素
7:     将  $x_k$  加入  $v$ 
8:     if  $v$  为最终解 then
9:       设置  $\text{flag} \leftarrow \text{true}$  并退出循环
10:    else if  $v$  为部分解 then
11:       $k \leftarrow k + 1$  {前进}
12:    end if
13:  end while
14:  重置  $X_k$  {通常将  $x_k \leftarrow 0$ , 表示将其重置}
15:   $k \leftarrow k - 1$  {回溯}
16: end while
17: if  $\text{flag}$  then
18:   输出  $v$ 
19: else
20:   输出“无解”
21: end if
```

常用剪枝函数：

- 用约束函数（如隐约束）在扩展节点处剪去不满足约束的子树；
- 用限界函数剪去得不到最优解的子树。

关于空间复杂性：

用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根节点到当前扩展节点的路径。

如果解空间树中从根节点到叶节点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 的空间。

如：规模为 n 的 0-1 背包问题的解空间大小为 $O(2^n)$ ， n 个顶点的 3 着色问题的解空间大小为 $O(3^n)$ ，

7.2 分支限界法

分支限界法 (Branch and Bound) 是一种用于解决组合优化问题的高效算法。它通过将问题的解空间划分为多个子空间，并使用优先级队列或堆来选择最有希望的子空间进行搜索，以剪枝无效的子空间，从而提高搜索效率。它也是一种在问题的解空间树中搜索问题解的算法。它类似于回溯法，但在求解目标和搜索方式上存在一些关键差异。

• 求解目标：

- 回溯法：回溯法主要关注的是找到**满足给定性质**的一个解或一组解。
例如，对于 3 着色问题或 8 皇后问题，回溯法会搜索所有可能的解，并找到满足特定条件的解。
- 分支限界法：分支限界法主要关注的是**目标函数的最大化或最小化**。
它通过设定一个上界（最大化问题）或下界（最小化问题）来进行搜索，以剪枝无效的子空间，并直接找到问题的最优解。

• 搜索方式：

- 回溯法：回溯法通常采用**深度优先搜索**的方式，即从根节点开始，一直向下搜索直到叶节点或无法继续扩展时回溯到上一层继续搜索。
- 分支限界法：分支限界法常采用**广（宽）度优先搜索或最小耗费（最大收益）优先**的方式进行搜索。
它在每一步都会生成当前节点的所有子节点，并根据约束条件和上界对子节点进行评估和选择，只保留有潜力的子节点加入活节点表中继续搜索。

分支限界法的基本思想如下所示：

- 分支限界法常以**广度优先**或以**最小耗费（最大效益）优先**的方式搜索问题的解空间树。
- 在分支限界法中，每一个活节点只有一次机会成为扩展节点。
活节点一旦成为扩展节点，就一次性产生其所有子节点。在这些子节点中，导致不可行解的子节点将被舍弃，其余子节点被加入活节点表（**队列**或是**优先队列**，详见后面的例子）中。
- 此后，从活节点表中取下一节点成为当前扩展节点，并重复上述节点扩展过程。这个过程一直持续到找到所需的解或活节点表为空时为止。

从活节点表中选择下一扩展节点的不同方式导致不同的分支限界法，常见的两种分支限界法有：

1. **队列式 (FIFO) 分支限界法 (宽度优先)**：按照队列**先进先出 (FIFO) 原则**选取下一个节点为扩展节点。
2. **优先队列式分支限界法 (最小耗费-[最小堆] 或是最大收益优先-[最大堆])**：按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。
 - 最大优先队列：使用最大堆，体现**最大收益**优先
 - 最小优先队列：使用最小堆，体现**最小耗费**优先

7.2.1 0-1 背包问题

考虑 $n=4$ 的背包问题（背包承重量为 $C=10$ ）：

物品	重量	价值	价值/重量
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

其中物品已经按照“价值-重量比”降序排列，这样会带来处理上的方便。

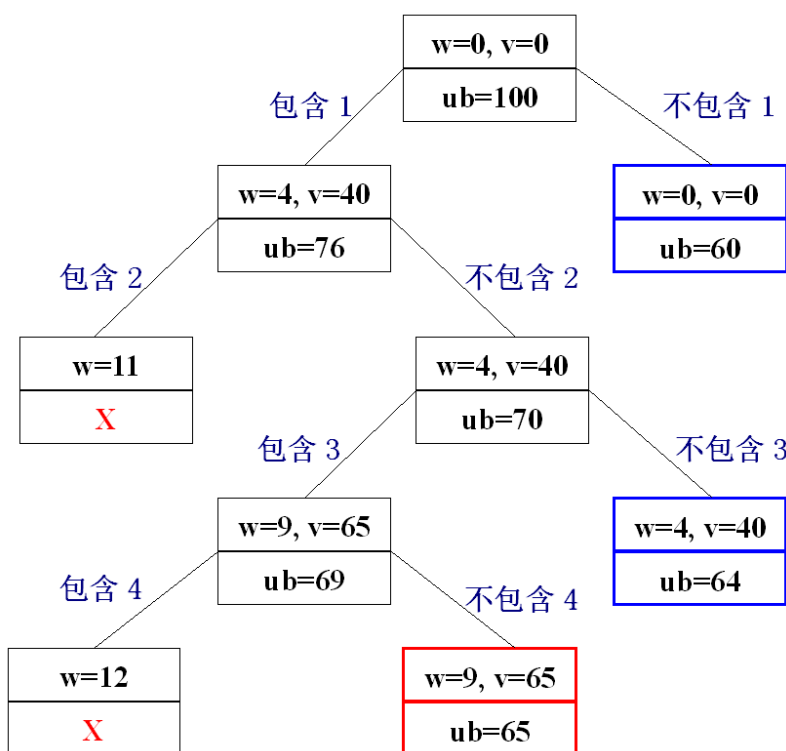
解决思路如下：

为每个搜索节点设置一个上界 ub (upper bound), ub 表示从该节点继续搜索，所获得的收益不会超过 ub 。设置该上界的一个简单方法是：把已经选择的物品总价值 v ，加上背包剩余承重量 $C - w$ (w 为已选物品的总重量) 与剩下可选择物品的最高价值-重量比的乘积，也即

$$ub = v + (C - w) \times \left(\frac{v_i}{w_i}\right)_{\max}$$

比如，若已装物品 1，则价值的上界为 $40 + (10 - 4) \times 6 = 76$

下图是构造的解空间树，每一层都会选择 ub 更大的解（如果解都存在的话）。



红框表示最终答案，蓝框是在队列中，但优先级不如最高 ub 的节点，此时若红框中的值小于蓝框中的值，需要扩展蓝框的搜索树。

7.2.2 分派问题

假设有 A, B, C, D 四位教师，每人都能教课程 I, II, III, IV 。但是各人备各门课所需要的时间不同，如下表所示：

	I	II	III	IV
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

现要求每门课有一个人教，并且每个人只能教一门课。问如何分配课程，才能使得总备课时间最少？

显然穷举法需要 $4!$ 种方案。试尝试使用分支限界法

假设一个人可以教多门课（一门课程只能是一个人上），这样得到的总备课时间为 T_0 ；那么可以肯定的是： $T \geq T_0$ 。

	I	II	III	IV
A	2	10	9	7
B	15	4	14	8
C	13	14	16	11
D	4	15	13	9

则最短备课时间 $T_0 = 2 + 4 + 9 + 7 = 22$

由此继续构建搜索树：

7.2.3 总结

方法	搜索方式	存储节点的常用数据结构	节点存储特性	常用应用
回溯法	深度优先搜索	堆栈	活节点的所有可行子节点被遍历后才被从栈中弹出	找出满足约束条件的所有解
分支限界法	广度优先或最小消耗（最大效益） 优先搜索	队列或优先队列（堆）	每个节点只有一次成为活节点的机会	找出满足约束条件的一个解或特定意义下的最优解

8 随机算法

8.1 随机算法的特点

经典确定性算法是指在解决问题时不使用随机性的算法。它们基于确定性的规则和逻辑，以一定的步骤和顺序执行，每次给出相同的输入时都产生相同的输出。这些算法通常被广泛应用于计算机科学和数学领域，并且具有**确定性、有限性、输入、输出**四大性质。

而对于随机算法，**可以不满足确定性**这一性质。

8.2 随机非重复采样问题

设有 n 个样本，要求从中随机选出 m 个样本 ($m < \frac{n}{2}$)。请设计一个随机算法来完成该任务，并分析该算法的时间复杂度。

思路：

1. 将 n 个样本存放于数组 $\text{Sample}[1 \dots n]$ 中。数组 $\text{Result}[1 \dots m]$ 存放选中的 m 个样本。
2. 定义一个长度为 n 的标志数组 $\text{Flag}[1 \dots n]$ 。
其中 $\text{Flag}[i] = \text{true}$ 表示对应位置的样本已经被选中；否则为未选中。
3. 随机生成一个落在区间 $[1, n]$ 的随机数 r 。
若 $\text{Flag}[r]$ 为 false，将 $\text{Sample}[r]$ 追加到数组 Result 中，并将 $\text{Flag}[r]$ 置为 true；若 $\text{Flag}[r]$ 为 true，则重新生成随机数 r ，直到 $\text{Flag}[r]$ 为 false。
4. 重复此步骤，直到 m 个样本选择完成。

Algorithm 25 随机非重复采样

Require: 样本数组 $\text{Sample}[1 \dots n]$, 选择样本的个数 m , 且 $m < \frac{n}{2}$

Ensure: 选中的样本 $\text{Result}[1 \dots m]$

```

1: for  $i \leftarrow 1$  to  $n$  do
2:    $\text{Flag}[i] \leftarrow \text{false}$ 
3: end for
4:  $k \leftarrow 0$ 
5: while  $k < m$  do
6:    $r \leftarrow \text{random}(1, n)$ 
7:   if not  $\text{Flag}[r]$  then
8:      $k \leftarrow k + 1$ 
9:      $\text{Result}[k] \leftarrow \text{Sample}[r]$ 
10:     $\text{Flag}[r] \leftarrow \text{true}$ 
11:   end if
12: end while
13: return  $\text{Result}[1 \dots m]$ 

```
